Multiprecision vs. Fixed-Width Digit Generation for JSON Canonicalization: An Empirical Study

Mark Lenhardt

Lattice Substrate

Abstract

JSON Canonicalization Scheme (RFC 8785) requires deterministic floating-point formatting that matches ECMA-262 `Number::toString`. We present the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a specification-constrained canonicalization pipeline, implemented in both Go and Rust to separate algorithmic effects from language/runtime effects. Our methodology gates performance measurement on conformance: only implementations passing 286,362 oracle vectors (generated from the V8 JavaScript engine) and 348 structural conformance cases (87 per implementation × 4 implementations) proceed to benchmarking. Using permutation tests, BCa bootstrap confidence intervals, and Benjamini–Hochberg FDR correction across 15 (CLI) and 10 (API) repeated measurements, we find that Schubfach is 1.3–1.7× faster than Burger–Dybvig on number-dense workloads ($\nu(D) \geq 40\%$ numeric content) in Go API benchmarks due to zero heap allocation in the digit-generation path; the advantage replicates at 1.6–4.2× in Rust, confirming it is algorithmic rather than language-specific (excluding two extreme outliers, `numeric-boundary` at 11.5× and `number-heavy` at 9.1× in Rust, discussed in §8). In end-to-end CLI measurement, 12 of 38 within-Go comparisons (19 canonicalize + 19 verify) reach BH-corrected significance (1.2–1.5× speedup); the remainder are dominated by process startup overhead. On string-dominant and deeply-nested inputs the difference narrows to <5%, and on several API-level non-number workloads (e.g., `control-escapes`, `rfc-key-sorting`) Burger–Dybvig is marginally faster (≤7%). All four implementations produce byte-identical output; Go binaries are additionally verified under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). We release the benchmark infrastructure, oracle vectors, and all raw data as a reproducibility artifact.

Keywords: JSON canonicalization, floating-point formatting, Schubfach, Burger–Dybvig, RFC 8785, ECMA-262, benchmark methodology, cross-language validation

1 Introduction

The JSON Canonicalization Scheme (RFC 8785) [15] standardizes a deterministic serialization of JSON values for use in digital signatures, content-addressable storage, and distributed consensus protocols. A critical requirement is that floating-point numbers must

be formatted identically to the output of ECMA-262 `Number::toString` [5], which specifies shortest-representation formatting under roundTiesToEven semantics.

Two families of algorithms can satisfy this requirement: multiprecision arithmetic approaches exemplified by Burger–Dybvig [3], and fixed-width integer approaches exemplified by Schubfach [8]. While these algorithms are individually well-studied, no prior work has empirically compared them *inside a specification-constrained canonicalization pipeline* where conformance is a hard prerequisite and the formatting stage is only one component of the total cost.

This paper makes three contributions:

1. The first empirical comparison of multiprecision vs. fixed-width digit generation under ECMA-262/RFC 8785 constraints, showing that Schubfach achieves 1.3–1.7× speedup on number-dense workloads ($\nu(D) \geq 40\%$) in API benchmarks; in end-to-end CLI measurement, the BH-significant comparisons show 1.2–1.5× speedup. String-dominant and deeply-nested inputs show <5% difference.

2. A benchmark methodology that gates performance measurement on conformance verification: implementations must pass 286,362 oracle vectors and 348 structural conformance cases before timing data is collected. We document noise floors and minimum observable effect sizes for all comparisons.

3. A cross-language validation design (2×2 matrix of algorithm × language) that separates algorithmic effects from language/runtime effects, with Go cross-compilation determinism verified for x86-64 and arm64 via QEMU user-mode emulation. All evidence is released as a reproducibility artifact.

2 Background

2.1 RFC 8785: JSON Canonicalization Scheme

RFC 8785 [15] defines a deterministic JSON serialization by specifying: (1) lexicographic key ordering using UTF-16 code unit comparison, (2) no insignificant whitespace, and (3) floating-point formatting matching ECMA-262 `Number::toString`.

The floating-point requirement is the most technically demanding. It requires producing the *shortest decimal representation* that round-trips to the original IEEE 754 binary64 value under roundTiesToEven, then applying the ECMA-262 formatting policy (exponential notation thresholds, sign handling, integer elision rules).

2.2 ECMA-262 Number Formatting

ECMA-262 [5] §6.1.6.1.20 specifies `Number::toString` as a two-stage process (formerly §7.1.12.1 in older editions):

1. **Digit generation**: find the shortest decimal significand d and exponent n such that $d \times 10^{n-k} = x$ (where k is the number of digits in d) and the value round-trips.

2. **Formatting**: apply branch logic based on the relationship between k (digit count) and n (decimal exponent position) to produce the final string. Key thresholds: $n \geq 1 \wedge n \leq 21$ produces fixed notation; $n \leq 0$ or $n > 21$ produces exponential notation.

2.3 IEEE 754 and Shortest Representation

IEEE 754-2019 [9] binary64 has a 52-bit significand, 11-bit exponent, and 1 sign bit. The *shortest representation* property requires finding the decimal string with fewest digits that uniquely identifies the original binary64 value among all representable values. This is the core algorithmic challenge addressed by both Burger–Dybvig and Schubfach.

3 Related Work

Extensive prior work has optimized float-to-string conversion in isolation. This paper is the first to evaluate these algorithms inside a specification-constrained canonicalization pipeline where conformance is a binary gate. We summarize the most relevant prior work, organized by the problem dimension each addresses.

3.1 Multiprecision Foundations

Burger and Dybvig [3] establish the theoretical framework for shortest-representation printing using arbitrary-precision rational arithmetic. Their algorithm is our multiprecision baseline. Gay's `dtoa.c` [6] provides the foundational practical implementation and remains widely used in language runtimes as a reference or fallback. Both works evaluate algorithms in isolation and do not consider performance inside a pipeline where formatting policy is externally constrained.

3.2 Fixed-Width Approaches

Loitsch [13] demonstrated that integer-only arithmetic suffices for the common case with Grisu. However, Grisu3 falls back to arbitrary precision for ∼0.5% of inputs, making conformance guarantees harder to establish in specification-constrained contexts.

Adams [1] proposed Ryū, achieving full coverage with fixed-width 128-bit arithmetic and lookup tables. Adams provides the most comprehensive prior performance comparison of fixed-width vs. multiprecision approaches (we refer the reader to Adams's literature review rather than replicating it), but evaluates algorithms in isolation on microbenchmarks without specification constraints.

Giulietti [8] introduced the Schubfach algorithm, which we implement and evaluate within the RFC 8785 pipeline. Jeon [10] extends and refines the Schubfach approach with Dragonbox, improving table compression and branch structure. Go 1.26 adopted Dragonbox for `strconv`, providing additional motivation for evaluating fixed-width approaches in specification-constrained contexts.

3.3 Comprehensive Experimental Review

Champagne Gareau and Lemire [4] provide the most comprehensive experimental comparison of modern float-to-string algorithms, evaluating Dragon4, Grisu3, Ryū, Schubfach, Dragonbox, and `std::to_chars` in C++. Their work evaluates algorithms in isolation on microbenchmarks. Our contribution differs: we evaluate two representatives (Burger–Dybvig and Schubfach) inside a specification-constrained canonicalization pipeline where (a) ECMA-262 formatting policy is a hard constraint, (b) correctness against 286,362 oracle vectors is a binary gate, and (c) digit generation is only one component of the total cost.

The complementary direction—fast string-to-float parsing—is addressed by Lemire [12], whose Eisel-Lemire algorithm achieves gigabyte-per-second parsing throughput. While outside our scope, this work shares the fixed-width arithmetic philosophy that motivates Schubfach and Dragonbox.

3.4 Deterministic Serialization Standards

Beyond JSON, deterministic encoding is required by other serialization formats. CBOR (RFC 8949) [2] specifies deterministic encoding rules (§4.2) for binary data, addressing analogous concerns about canonical representation. W3C RDF Dataset Canonicalization [16] addresses graph-level canonicalization (structural equivalence of RDF datasets), a fundamentally different problem from the byte-level serialization determinism required by RFC 8785. Our conformance-gated methodology could be adapted to other deterministic serialization benchmarks.

3.5 JSON Canonicalization

RFC 8785 [15] is the primary standard. Prior to RFC 8785, ad-hoc canonicalization schemes existed in various ecosystems (e.g., JSON-LD canonicalization, OLPC canonical JSON). None specify the floating-point formatting requirement with the precision of ECMA-262 [5].

3.6 Benchmark Methodology

Our conformance-before-performance approach draws on the principle that correctness is a prerequisite for meaningful performance comparison. Georges et al. [7] established rigorous statistical methods for Java performance evaluation, advocating confidence intervals and effect sizes over single-point estimates. Mytkowicz et al. [14] demonstrated that measurement bias (link order, environment size) can reverse performance conclusions, motivating our randomized task ordering and environment documentation. Kalibera and Jones [11] formalized the analysis of multi-level experimental designs in benchmarking. Our use of permutation tests and BCa bootstrap confidence intervals follows this modern statistical practice for performance evaluation.

4 Algorithms

4.1 Definitions

Definition 1 (Minimal Significand)**.** Given a finite IEEE 754 binary64 value x, the *minimal significand* is the shortest decimal significand d (with digit count k) such that parsing $d \times 10^{n-k}$ under roundTiesToEven recovers x exactly.

Definition 2 (Minimal Printed String)**.** The *minimal printed string* of x is the output of ECMA-262 `Number::toString` applied to x: the minimal significand formatted according to the ECMA-262 branch table.

Remark 1. Definitions 1 and 2 are not equivalent in string length. Consider $x = 10^{20}$: the minimal significand is $d = 1$ ($k = 1$, $n = 21$). Since $n = 21 \leq 21$, ECMA-262 uses fixed notation, yielding `100000000000000000000` (21 characters). Now consider $x = 10^{21}$: again $d = 1$ ($k = 1$, $n = 22$), but $n > 21$ triggers exponential notation, yielding `1e+21` (5 characters). The $n = 21$ boundary creates divergence between significand minimality and printed-string length. This means implementations cannot optimize for shorter output strings because the ECMA-262 formatting is spec-mandated.

4.2 Burger–Dybvig (Multiprecision)

The Burger–Dybvig algorithm [3] uses arbitrary-precision rational arithmetic to compute the shortest decimal representation. It maintains big-integer numerator/denominator pairs and iteratively extracts decimal digits. The algorithm is general (handles any radix) but requires heap-allocated multiprecision integers, with cost proportional to the magnitude of the exponent.

In our Go implementation, this uses `math/big.Int` operations. The `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers across calls.

4.3 Schubfach (Fixed-Width)

The Schubfach algorithm [8] uses fixed-width 64/128-bit arithmetic, relying on `math/bits` for 128-bit multiplication (`Mul64`, `Add64`) and bit inspection (`TrailingZeros64`). It uses precomputed tables of powers of ten and 128-bit multiplications to determine the shortest decimal significand. No heap allocation is required in the digit-generation path.

The key advantage is constant-time (with respect to exponent magnitude) operation and zero GC pressure. The precomputed table is approximately 5 KiB.

4.4 ECMA-262 Formatting Stage

Both algorithms share an identical formatting stage that converts the minimal significand (d, k, n) triple into the final ECMA-262-compliant string. This stage implements the branch table from ECMA-262 §6.1.6.1.20 and is pure string manipulation with no arithmetic.

5 Implementation

We maintain four implementations with a common CLI ABI:

- `impl-schubfach` (Go): Schubfach number formatting.
- `impl-json-canon` (Go): Burger–Dybvig number formatting.
- `impl-schubfach-rs` (Rust): Schubfach-aligned Rust path.
- `impl-json-canon-rs` (Rust): Burger–Dybvig-aligned Rust path.

All four implementations contain their own digit-generation code and do not delegate to the standard library's float-to-string routines (e.g., Go's `strconv.FormatFloat` or Rust's `std::fmt`). This is necessary because the standard routines do not implement ECMA-262 formatting policy. Consequently, Go 1.26's adoption of Dragonbox for `strconv` does not affect our measurements.

The lab harness treats each implementation as a black-box canonicalizer with identical CLI contracts (`canonicalize`, `verify`, exit-code policy, and error-class expectations). This preserves algorithm comparability while allowing language/runtime differences to be measured independently.

Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`) to align optimization policy with production benchmarking practice.

5.1 Rust Implementation Notes

The Rust Schubfach implementation uses native `u128` arithmetic for the 128-bit intermediate products required by the algorithm, eliminating the need for compiler intrinsics or multi-word emulation. The Rust Burger–Dybvig implementation uses `num-bigint::BigInt` v0.4.6 for multiprecision arithmetic; buffer lifecycle is managed by Rust's RAII ownership model (deterministic `Drop`), removing the need for explicit pooling strategies such as Go's `sync.Pool`. Cargo.lock files are committed for both Rust implementations to ensure reproducible dependency resolution.

Both Rust implementations share an identical JSON parser and serializer; only the `jcsfloat` digit-generation module differs between them, mirroring the Go design. Rust binaries are approximately 430–456 KB (stripped), compared to 2.5–2.7 MB for the statically-linked Go binaries.

6 Methodology

6.1 Conformance-Before-Performance

Our methodology enforces a strict ordering: conformance verification must pass before any performance data is collected. This prevents measuring the speed of incorrect implementations.

Conformance gate: 87 structural test cases per implementation (348 total across 4 implementations) covering key ordering, Unicode handling, numeric formatting, escape

sequences, depth limits, and invalid input rejection. All implementations must produce byte-identical output (verified via SHA-256) and identical error classification.

Oracle vectors: 286,362 IEEE 754 binary64 values with expected `String(x)` output generated by the V8 JavaScript engine (Node.js v23.3.0, V8 12.9.202.28). (2,153 values appear in both vector files as overlapping boundary coverage; the total counts file lines, with 284,209 unique values.) Coverage includes all subnormals with bit patterns $\leq 2^{12}$ (4,096 of approximately 4.5 million total subnormals, providing dense coverage near zero where formatting edge cases concentrate), subnormal/normal boundary (± 128 around the transition), all powers of two in the binary64 exponent range, RFC 8785 Appendix B examples with ± 2 neighbors, and pseudorandom fill.

6.2 Benchmark Design

CLI benchmarks: end-to-end process execution measuring wall time, CPU time (user + system), and peak RSS. 15 repetitions per implementation/mode/workload combination, with 3 warmup runs discarded. Task order is randomized with a fixed seed for reproducibility.

Cross-language comparison pairs: each algorithm is implemented in Go and Rust, yielding four implementations: Go-Schubfach, Go-Burger–Dybvig, Rust-Schubfach, Rust-Burger–Dybvig. These are compared via four pairwise tests: two within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd) and two cross-language pairs (schubfach:go-vs-rs, bd:go-vs-rs). The within-language pairs test the algorithmic effect while controlling for language/runtime; the cross-language pairs test language/runtime effects while controlling for algorithm. Directional consistency across both within-language pairs provides evidence that the algorithmic advantage is not language-specific, though this is not a formal interaction test (see §8). The Rust Burger–Dybvig implementation uses `num-bigint::BigInt` (heap-allocated multiprecision integers with RAII lifecycle, no garbage collector), making it a structurally analogous but runtime-different counterpart to the Go `math/big`-based path. Rust builds use release configuration (`opt-level=3`, `lto=true`, `codegen-units=1`) to align optimization policy with production benchmarking practice.

API benchmarks: Go `testing.B` microbenchmarks (Go implementations) and a Rust native benchmark harness (Rust implementations) measuring canonicalization and verify paths directly, 10 iterations per benchmark. Go's `testing.B` auto-calibrates the number of inner iterations per sample (typically hundreds to millions), whereas the Rust harness times each function call individually (N=1); both report ns/op. The Rust harness includes 3 warmup iterations (discarded) before measurement to stabilize caches and lazy-initialized data structures. The Rust harness emits Go-compatible `Benchmark... ns/op` lines so the same statistical pipeline can be applied uniformly.

Measurement granularity note: The Go and Rust API harnesses differ in measurement granularity. Go's `testing.B` averages over many inner iterations per sample, producing low-variance samples. The Rust harness reports single-call latency, producing higher-variance samples. Within-language comparisons are unaffected since both implementations in a pair use the same harness. Cross-language API comparisons should be interpreted with this asymmetry in mind; we report them for completeness but emphasize within-language results for algorithmic conclusions.

Differential fuzzing: 2,000 randomly-generated JSON inputs processed by all implementations, verifying byte-identical output.

6.3 Workload Characterization

Definition 3 (Number value fraction)**.** For a JSON document D, define $\nu(D)$ as the fraction of JSON values that are numbers. Array and object containers are counted as values; their elements and members are counted separately. Formally, $\nu(D) = |\{v \in V(D) : v \text{ is a JSON number}\}|/|V(D)|$, where $V(D)$ is the multiset of all JSON values in D (including containers).

Table 1 reports the structural properties of each benchmark workload, including $\nu(D)$, total value count, maximum nesting depth, and unique key count.

Table 1: Workload structural characterization.

Workload	Bytes	Total Values	Numbers	$\nu(D)$ (%)	Max Depth	Keys
array-2048	98,901	10,243	4,096	40.0	3	8,194
array-256	11,914	1,283	512	39.9	3	1,026
canonical-minimal	7	2	1	50.0	1	1
control-escapes	59	4	0	0.0	1	3
deep	1,158	141	0	0.0	140	140
deep-64	510	65	0	0.0	64	64
escaped-key-order	18	3	2	66.7	1	2
large	463,537	40,963	16,384	40.0	3	32,770
long-string	16,403	3	0	0.0	1	2
medium	13,740	1,283	512	39.9	3	1,026
mixed-prod	5,653	458	128	27.9	4	393
nested-mixed	49	10	4	40.0	3	7
number-heavy	140	13	10	76.9	2	4
numeric-boundary	98	9	7	77.8	2	2
rfc-key-sorting	202	8	0	0.0	1	7
small	45	5	2	40.0	1	4
surrogate-pair	29	3	0	0.0	1	2
unicode	98	4	0	0.0	1	3
verify-whitespace	24	5	3	60.0	2	2

Total Values counts containers (arrays, objects) and their elements separately.
$\nu(D)$ = number values / total values $\times$ 100.

6.4 Statistical Protocol

All comparisons use:

- Permutation tests (5,000 resamples) for significance at $\alpha = 0.05$.
- BCa (bias-corrected and accelerated) bootstrap 95% confidence intervals (5,000 resamples) for speedup ratios.

- Cohen's d for effect size.[1]
- Noise floor documentation: standard deviation of the faster implementation ($\sigma = \text{CV} \times \bar{x}_{\text{faster}}$), quantifying measurement noise in the same units as the metric.
- Minimum detectable effect (MDE): $\frac{2.8\sqrt{2} \times \text{CV} \times 100}{\sqrt{N}}$, where $2.8 \approx z_{\alpha/2} + z_\beta$ for $\alpha = 0.05$ and 80% power (approximate MDE for a two-sample z-test with equal group sizes; uses only the faster implementation's CV and n as an approximation). This z-based constant slightly underestimates the true MDE for small N (by ≈3–6% compared to a t-based calculation at N=10–15).
- Benjamini-Hochberg false discovery rate (FDR) control at $q = 0.05$ across all 317 comparisons (152 CLI end-to-end + 165 API). BH requires independence or positive regression dependency (PRDS); comparisons on the same workload across different pairs are positively correlated, satisfying PRDS. We report both raw and BH-adjusted p-values.
- No outlier exclusion. Permutation tests are nonparametric and robust to non-normality; BCa bootstrap accounts for skewness via the acceleration constant.
- Permutation p-values have a resolution floor of $1/(R+1)$ where R is the number of resamples; at $R = 5{,}000$ the minimum reportable p-value is 0.0002. Values reported at this floor should be interpreted as $p < 0.0002$.

Statistical resampling uses deterministic per-comparison seeds derived via FNV-1a hashing of the comparison key, ensuring reproducible p-values and confidence intervals across runs.

6.5 Environment

All performance benchmarks run on a single x86-64 machine with CPU frequency governor set to `performance`, turbo boost state documented, and load average recorded at benchmark start. Full hardware specification (CPU model, cache hierarchy, RAM, relevant CPU feature flags) is captured automatically by the benchmark harness.

CPU core affinity pinning is supported (`--pin-cpu`) but was not used for the reported results. The i7-12700K has heterogeneous P-core/E-core topology; without pinning, the OS scheduler may migrate processes. This adds variance but does not bias the comparison since both implementations share the same scheduling environment.

Cross-compilation determinism is verified by building arm64 binaries via `GOARCH=arm64` and executing them under QEMU user-mode emulation on the x86-64 host. QEMU user-mode translates system calls but executes floating-point operations on the host CPU; this tests Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 performance benchmarks are deferred to future work on bare-metal hardware.

[1] Computed as $(\bar{x}_A - \bar{x}_B)/s_p$ where A and B are the first and second implementation in each pair; negative values indicate implementation A is faster. For within-language pairs (go:schubfach-vs-bd, rs:schubfach-vs-bd), A is the Schubfach variant. For cross-language pairs, A is the Go implementation.

7 Results

7.1 Conformance

All four implementations (Go and Rust × Schubfach and Burger–Dybvig) pass the full conformance matrix: 348 total cases (87 structural tests × 4 implementations, 0 failures). Output SHA-256 digests are byte-identical across all implementations for valid inputs.

7.2 API Benchmarks

API benchmarks isolate the algorithmic cost of the canonicalization function, removing process startup and I/O overhead. These measurements most directly reflect the digit-generation performance difference.

Table 2 summarizes the key API results for the within-language algorithm comparisons (canonicalize mode). Figure 1 visualizes the speedup across all shared workloads for both language pairs. Full results for all pairs, modes, and workloads appear in Appendix A.

Table 3 presents the within-language API comparisons with BH-corrected p-values, confidence intervals, and effect sizes.

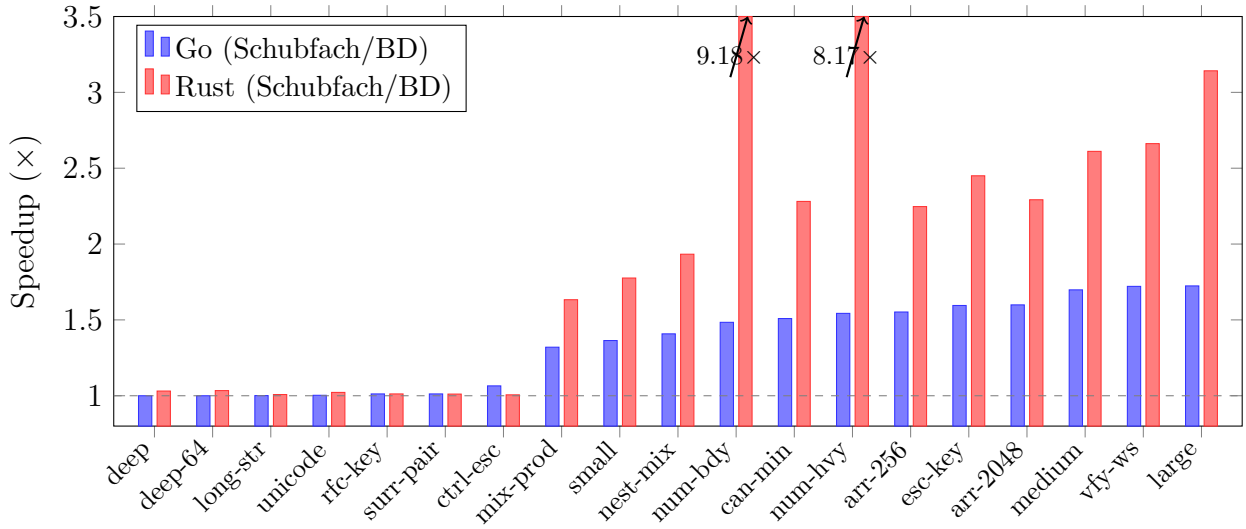

Figure 1: API benchmark speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode). Workloads sorted by Go speedup. Values above the dashed line indicate Schubfach is faster. The Rust `numeric-boundary` and `number-heavy` bars are truncated at 3.5×; actual values are 9.18× and 8.17× respectively (see §8). Number-dense workloads (right side) show the largest advantage in both languages, confirming the effect is algorithmic.

7.3 CLI Benchmarks

CLI benchmarks capture end-to-end pipeline cost including process startup, argument parsing, file I/O, and output serialization. Estimated process startup overhead (approximately

Table 2: API benchmark summary: within-language algorithm comparisons (canonicalize mode, mean of n=10).

Pair	Workload	Schubfach (ns/op)	Burger–Dybvig (ns/op)	Speedup
go:schubfach-vs-bd	array-2048	2542678	4074092	1.60×
go:schubfach-vs-bd	array-256	305339	470900	1.54×
go:schubfach-vs-bd	control-escapes	1343	1252	1.07×
go:schubfach-vs-bd	deep	36457	36492	1.00×
go:schubfach-vs-bd	deep-64	16200	16166	1.00×
go:schubfach-vs-bd	large	11949790	20563068	1.72×
go:schubfach-vs-bd	long-string	76630	75884	1.01×
go:schubfach-vs-bd	medium	341640	580994	1.70×
go:schubfach-vs-bd	mixed-prod	136144	179680	1.32×
go:schubfach-vs-bd	number-heavy	13814	21295	1.54×
go:schubfach-vs-bd	numeric-boundary	12652	18641	1.47×
go:schubfach-vs-bd	rfc-key-sorting	3006	2965	1.01×
go:schubfach-vs-bd	small	1240	1668	1.35×
go:schubfach-vs-bd	unicode	1553	1548	1.00×
go:schubfach-vs-bd	verify-whitespace	910	1562	1.72×
rs:schubfach-vs-bd	array-2048	1368742	3973980	2.90×
rs:schubfach-vs-bd	array-256	172566	440236	2.55×
rs:schubfach-vs-bd	control-escapes	1158	640	1.81×
rs:schubfach-vs-bd	deep	31947	30577	1.04×
rs:schubfach-vs-bd	deep-64	12916	11102	1.16×
rs:schubfach-vs-bd	large	5652514	23450172	4.15×
rs:schubfach-vs-bd	long-string	18825	17012	1.11×
rs:schubfach-vs-bd	medium	185816	632470	3.40×
rs:schubfach-vs-bd	mixed-prod	68092	138742	2.04×
rs:schubfach-vs-bd	number-heavy	1588	14419	9.08×
rs:schubfach-vs-bd	numeric-boundary	978	11247	11.50×
rs:schubfach-vs-bd	rfc-key-sorting	1311	1340	1.02×
rs:schubfach-vs-bd	small	655	1254	1.92×
rs:schubfach-vs-bd	unicode	1491	1558	1.05×
rs:schubfach-vs-bd	verify-whitespace	406	1352	3.33×

15 of 19 workloads shown; 4 very small workloads omitted for space (canonical-minimal, escaped-key-order, nested-mixed, surrogate-pair). Full results for all pairs, modes, and workloads in Appendix A.

Table 3: Within-language API comparisons (canonicalize mode; BH-significant marked with *).

Pair	Workload	Winner	Speedup	CI_{95}	p_{adj}	d	MDE
go:schubfach-vs-bd	array-2048	schubfach*	1.60×	[1.595, 1.608]	0.0003	-92.35	
rs:schubfach-vs-bd	array-2048	schubfach-rs*	2.90×	[2.855, 2.931]	0.0003	-101.90	
go:schubfach-vs-bd	array-256	schubfach*	1.54×	[1.531, 1.550]	0.0003	-61.98	
rs:schubfach-vs-bd	array-256	schubfach-rs*	2.55×	[2.478, 2.600]	0.0003	-50.86	
go:schubfach-vs-bd	canonical-minimal	schubfach*	1.52×	[1.508, 1.539]	0.0003	-30.19	
rs:schubfach-vs-bd	canonical-minimal	schubfach-rs*	1.61×	[1.466, 1.898]	0.0003	-3.80	
go:schubfach-vs-bd	control-escapes	json-canon*	1.07×	[1.065, 1.079]	0.0003	8.65	
rs:schubfach-vs-bd	control-escapes	json-canon-rs*	1.81×	[1.514, 1.914]	0.0005	4.27	
go:schubfach-vs-bd	deep-64	json-canon	1.00×	[0.997, 1.007]	0.5062	0.33	
rs:schubfach-vs-bd	deep-64	json-canon-rs*	1.16×	[1.097, 1.191]	0.0003	3.34	
go:schubfach-vs-bd	deep	schubfach	1.00×	[0.995, 1.007]	0.7903	-0.13	
rs:schubfach-vs-bd	deep	json-canon-rs*	1.04×	[1.016, 1.083]	0.0183	1.13	
go:schubfach-vs-bd	escaped-key-order	schubfach*	1.60×	[1.591, 1.609]	0.0003	-64.73	
rs:schubfach-vs-bd	escaped-key-order	schubfach-rs*	1.55×	[1.444, 1.792]	0.0003	-4.54	
go:schubfach-vs-bd	large	schubfach*	1.72×	[1.714, 1.728]	0.0003	-101.13	
rs:schubfach-vs-bd	large	schubfach-rs*	4.15×	[4.119, 4.177]	0.0003	-137.80	
go:schubfach-vs-bd	long-string	json-canon	1.01×	[0.997, 1.023]	0.1999	0.63	
rs:schubfach-vs-bd	long-string	json-canon-rs*	1.11×	[1.071, 1.122]	0.0003	4.11	
go:schubfach-vs-bd	medium	schubfach*	1.70×	[1.690, 1.712]	0.0003	-62.02	
rs:schubfach-vs-bd	medium	schubfach-rs*	3.40×	[3.129, 3.505]	0.0003	-39.04	
go:schubfach-vs-bd	mixed-prod	schubfach*	1.32×	[1.313, 1.333]	0.0003	-29.25	
rs:schubfach-vs-bd	mixed-prod	schubfach-rs*	2.04×	[2.014, 2.063]	0.0003	-58.00	
go:schubfach-vs-bd	nested-mixed	schubfach*	1.41×	[1.402, 1.419]	0.0003	-43.96	
rs:schubfach-vs-bd	nested-mixed	schubfach-rs*	2.19×	[2.114, 2.249]	0.0003	-23.61	
go:schubfach-vs-bd	number-heavy	schubfach*	1.54×	[1.525, 1.553]	0.0003	-44.59	
rs:schubfach-vs-bd	number-heavy	schubfach-rs*	9.08×	[7.899, 9.885]	0.0008	-59.55	
go:schubfach-vs-bd	numeric-boundary	schubfach*	1.47×	[1.466, 1.483]	0.0003	-66.14	
rs:schubfach-vs-bd	numeric-boundary	schubfach-rs*	11.50×	[11.355, 11.754]	0.0003	-175.11	
go:schubfach-vs-bd	rfc-key-sorting	json-canon*	1.01×	[1.009, 1.019]	0.0005	2.23	
rs:schubfach-vs-bd	rfc-key-sorting	schubfach-rs	1.02×	[1.000, 1.054]	0.1569	-0.70	
go:schubfach-vs-bd	small	schubfach*	1.35×	[1.338, 1.351]	0.0003	-49.47	
rs:schubfach-vs-bd	small	schubfach-rs*	1.92×	[1.870, 1.962]	0.0003	-19.05	
go:schubfach-vs-bd	surrogate-pair	json-canon*	1.01×	[1.008, 1.018]	0.0005	2.23	
rs:schubfach-vs-bd	surrogate-pair	json-canon-rs	1.02×	[0.983, 1.061]	0.5062	0.35	
go:schubfach-vs-bd	unicode	json-canon	1.00×	[0.998, 1.007]	0.2335	0.59	
rs:schubfach-vs-bd	unicode	schubfach-rs*	1.05×	[1.027, 1.065]	0.0008	-2.02	
go:schubfach-vs-bd	verify-whitespace	schubfach*	1.72×	[1.709, 1.726]	0.0003	-75.40	
rs:schubfach-vs-bd	verify-whitespace	schubfach-rs*	3.33×	[3.224, 3.430]	0.0003	-30.09	

*BH-significant at $q = 0.05$. d = Cohen's d $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster.
MDE = minimum detectable effect at 80% power. Full 317-comparison report in Appendix A.

1.3 ms for Go, 0.5 ms for Rust) dominates for smaller workloads, attenuating the algorithmic signal observed in API benchmarks.

Table 4 summarizes the key CLI results. Figure 2 shows the end-to-end speedup with BH-significance markers. Full results appear in Appendix A.

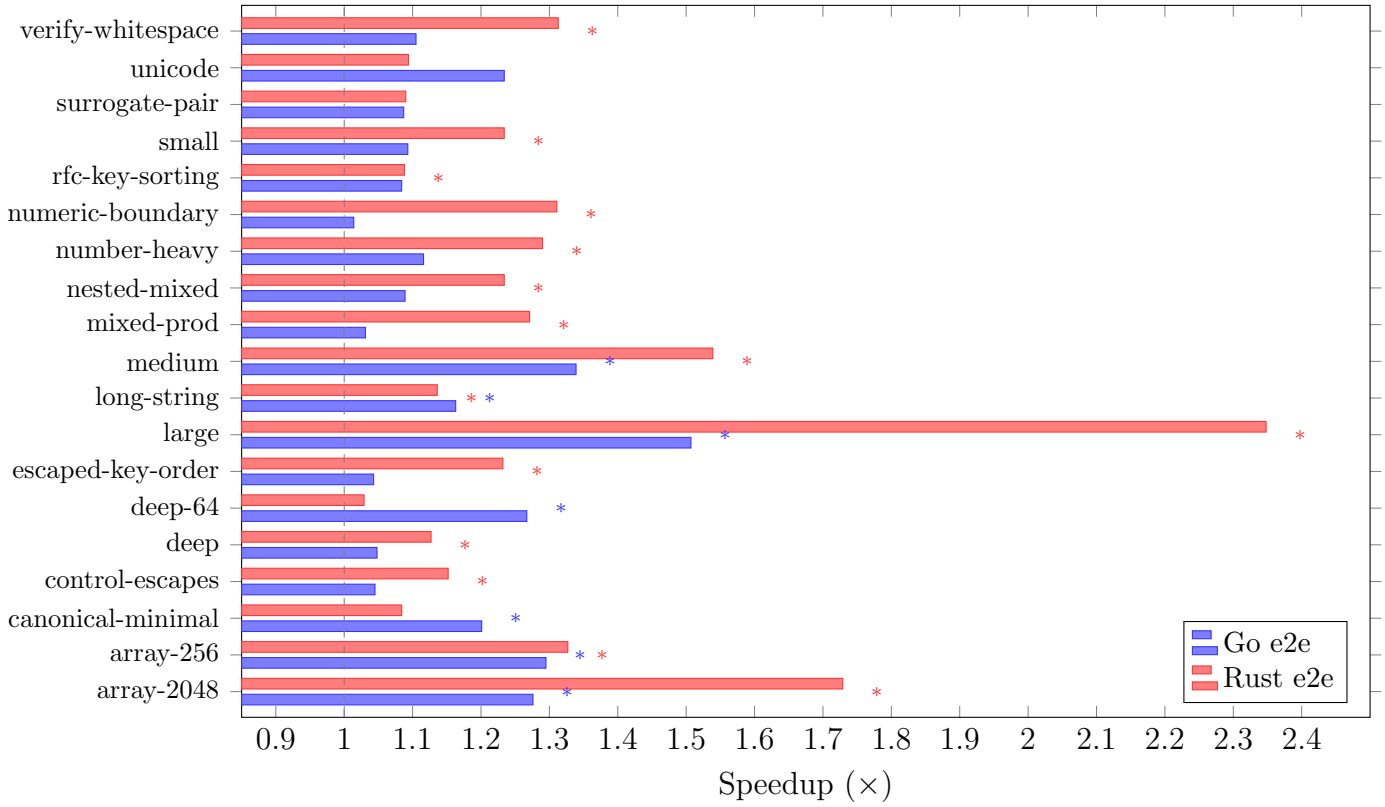

Figure 2: CLI end-to-end speedup of Schubfach over Burger–Dybvig by workload (canonicalize mode, 15 repetitions each). Asterisks (∗) mark BH-significant comparisons ($q < 0.05$). The Rust within-language pair achieves significance on 15 of 19 workloads; the Go pair on 7 of 19 workloads (canonicalize mode). Process startup overhead (≈1.3 ms Go, ≈0.5 ms Rust) attenuates the algorithmic signal more strongly in Go.

7.4 Statistical Comparisons

Table 5 presents the within-language CLI comparisons (canonicalize mode) with BH-corrected p-values. The full 317-comparison statistical report appears in Appendix A.

7.5 Cross-Language Validation

All four implementations pass the full conformance matrix (348 total cases: 87 structural tests × 4 implementations, 0 failures) and produce byte-identical output on all oracle vectors. 2,000 differential fuzz cases yield zero cross-implementation divergences.

The statistical report includes four comparison classes:

Table 4: CLI benchmark summary: within-language comparisons (canonicalize mode, n=15, 3 warmup discarded).

Pair	Workload	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	5.317	7.486	1.41×	schubfach
go:schubfach-vs-bd	array-256	2.285	2.367	1.04×	schubfach
go:schubfach-vs-bd	canonical-minimal	1.630	1.452	1.12×	json-canon
go:schubfach-vs-bd	control-escapes	1.639	1.588	1.03×	json-canon
go:schubfach-vs-bd	deep	1.596	1.769	1.11×	schubfach
go:schubfach-vs-bd	deep-64	1.498	1.679	1.12×	schubfach
go:schubfach-vs-bd	escaped-key-order	1.552	1.676	1.08×	schubfach
go:schubfach-vs-bd	large	16.427	24.928	1.52×	schubfach
go:schubfach-vs-bd	long-string	1.595	1.787	1.12×	schubfach
go:schubfach-vs-bd	medium	2.113	2.657	1.26×	schubfach
go:schubfach-vs-bd	mixed-prod	1.688	1.879	1.11×	schubfach
go:schubfach-vs-bd	nested-mixed	1.519	1.603	1.06×	schubfach
go:schubfach-vs-bd	number-heavy	1.476	1.756	1.19×	schubfach
go:schubfach-vs-bd	numeric-boundary	1.420	1.706	1.20×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	1.482	1.448	1.02×	json-canon
go:schubfach-vs-bd	small	1.467	1.551	1.06×	schubfach
go:schubfach-vs-bd	surrogate-pair	1.499	1.605	1.07×	schubfach
go:schubfach-vs-bd	unicode	1.549	1.593	1.03×	schubfach
go:schubfach-vs-bd	verify-whitespace	1.549	1.627	1.05×	schubfach
rs:schubfach-vs-bd	array-2048	2.611	4.644	1.78×	schubfach-rs
rs:schubfach-vs-bd	array-256	0.824	1.192	1.45×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	0.561	0.642	1.14×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	0.600	0.615	1.02×	schubfach-rs
rs:schubfach-vs-bd	deep	0.621	0.705	1.14×	schubfach-rs
rs:schubfach-vs-bd	deep-64	0.596	0.661	1.11×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	0.547	0.696	1.27×	schubfach-rs
rs:schubfach-vs-bd	large	7.562	18.613	2.46×	schubfach-rs
rs:schubfach-vs-bd	long-string	0.631	0.667	1.06×	schubfach-rs
rs:schubfach-vs-bd	medium	0.891	1.402	1.57×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	0.679	0.893	1.32×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	0.554	0.669	1.21×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	0.542	0.695	1.28×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	0.543	0.741	1.36×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	0.567	0.628	1.11×	schubfach-rs
rs:schubfach-vs-bd	small	0.591	0.674	1.14×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	0.559	0.622	1.11×	schubfach-rs
rs:schubfach-vs-bd	unicode	0.581	0.636	1.09×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	0.549	0.700	1.27×	schubfach-rs

Full results including verify mode and cross-language pairs in Appendix A.

Table 5: Within-language CLI comparisons (canonicalize mode; BH-significant marked with *).

Pair	Workload	Winner	Speedup	CI_{95}	p_{adj}	d	MDE (%)
go:schubfach-vs-bd	array-2048	schubfach*	1.41×	[1.310, 1.527]	0.0003	-2.98	10.1
rs:schubfach-vs-bd	array-2048	schubfach-rs*	1.78×	[1.589, 2.010]	0.0003	-3.30	18.9
go:schubfach-vs-bd	array-256	schubfach	1.04×	[0.948, 1.150]	0.5205	-0.25	13.6
rs:schubfach-vs-bd	array-256	schubfach-rs*	1.45×	[1.306, 1.593]	0.0003	-2.55	13.2
go:schubfach-vs-bd	canonical-minimal	json-canon	1.12×	[0.986, 1.266]	0.1068	0.65	21.0
rs:schubfach-vs-bd	canonical-minimal	schubfach-rs*	1.14×	[1.087, 1.216]	0.0003	-1.72	9.5
go:schubfach-vs-bd	control-escapes	json-canon	1.03×	[0.912, 1.174]	0.6663	0.17	21.1
rs:schubfach-vs-bd	control-escapes	schubfach-rs	1.02×	[0.941, 1.091]	0.5663	-0.23	9.6
go:schubfach-vs-bd	deep-64	schubfach	1.12×	[0.988, 1.267]	0.1261	-0.62	18.1
rs:schubfach-vs-bd	deep-64	schubfach-rs*	1.11×	[1.030, 1.215]	0.0194	-0.86	6.9
go:schubfach-vs-bd	deep	schubfach	1.11×	[0.960, 1.276]	0.2013	-0.50	21.3
rs:schubfach-vs-bd	deep	schubfach-rs	1.14×	[1.019, 1.268]	0.0511	-0.79	12.8
go:schubfach-vs-bd	escaped-key-order	schubfach	1.08×	[0.960, 1.235]	0.2825	-0.43	18.4
rs:schubfach-vs-bd	escaped-key-order	schubfach-rs*	1.27×	[1.214, 1.348]	0.0003	-3.44	9.9
go:schubfach-vs-bd	large	schubfach*	1.52×	[1.485, 1.549]	0.0003	-12.68	2.9
rs:schubfach-vs-bd	large	schubfach-rs*	2.46×	[2.360, 2.540]	0.0003	-20.08	6.8
go:schubfach-vs-bd	long-string	schubfach	1.12×	[0.983, 1.261]	0.1216	-0.62	17.1
rs:schubfach-vs-bd	long-string	schubfach-rs	1.06×	[0.981, 1.123]	0.1305	-0.59	9.4
go:schubfach-vs-bd	medium	schubfach*	1.26×	[1.104, 1.439]	0.0039	-1.20	22.2
rs:schubfach-vs-bd	medium	schubfach-rs*	1.57×	[1.451, 1.731]	0.0003	-3.61	14.1
go:schubfach-vs-bd	mixed-prod	schubfach	1.11×	[0.974, 1.291]	0.1781	-0.52	20.4
rs:schubfach-vs-bd	mixed-prod	schubfach-rs*	1.32×	[1.215, 1.440]	0.0003	-2.27	13.2
go:schubfach-vs-bd	nested-mixed	schubfach	1.06×	[0.917, 1.223]	0.5205	-0.26	19.3
rs:schubfach-vs-bd	nested-mixed	schubfach-rs*	1.21×	[1.102, 1.320]	0.0018	-1.36	9.4
go:schubfach-vs-bd	number-heavy	schubfach*	1.19×	[1.048, 1.341]	0.0174	-1.00	18.8
rs:schubfach-vs-bd	number-heavy	schubfach-rs*	1.28×	[1.174, 1.412]	0.0003	-1.92	13.2
go:schubfach-vs-bd	numeric-boundary	schubfach*	1.20×	[1.044, 1.353]	0.0176	-0.97	18.3
rs:schubfach-vs-bd	numeric-boundary	schubfach-rs*	1.36×	[1.272, 1.461]	0.0003	-3.11	11.0
go:schubfach-vs-bd	rfc-key-sorting	json-canon	1.02×	[0.886, 1.181]	0.7778	0.11	20.1
rs:schubfach-vs-bd	rfc-key-sorting	schubfach-rs*	1.11×	[1.006, 1.207]	0.0486	-0.78	9.7
go:schubfach-vs-bd	small	schubfach	1.06×	[0.922, 1.214]	0.4886	-0.28	21.9
rs:schubfach-vs-bd	small	schubfach-rs*	1.14×	[1.062, 1.211]	0.0023	-1.35	7.3
go:schubfach-vs-bd	surrogate-pair	schubfach	1.07×	[0.948, 1.212]	0.3303	-0.39	16.9
rs:schubfach-vs-bd	surrogate-pair	schubfach-rs*	1.11×	[1.038, 1.189]	0.0084	-1.09	10.0
go:schubfach-vs-bd	unicode	schubfach	1.03×	[0.898, 1.157]	0.6899	-0.16	15.1
rs:schubfach-vs-bd	unicode	schubfach-rs*	1.09×	[1.024, 1.170]	0.0238	-0.93	8.9
go:schubfach-vs-bd	verify-whitespace	schubfach	1.05×	[0.933, 1.183]	0.4840	-0.28	17.3
rs:schubfach-vs-bd	verify-whitespace	schubfach-rs*	1.27×	[1.202, 1.365]	0.0003	-2.74	11.2

*BH-significant at $q = 0.05$. d = Cohen's d $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster.
MDE = minimum detectable effect at 80% power. Full 317-comparison report in Appendix A.

- within-Go (Schubfach vs. Burger–Dybvig),
- within-Rust (Schubfach-rs vs. Burger–Dybvig-rs),
- same-algorithm Go-vs-Rust for Schubfach,
- same-algorithm Go-vs-Rust for Burger–Dybvig.

In CLI end-to-end measurement, Rust implementations are approximately 2–3× faster than their Go counterparts, primarily due to lower process startup overhead (Rust native binary vs. Go runtime initialization). In API benchmarks, Rust-Schubfach is faster than Rust-Burger–Dybvig on number-dense workloads, replicating the within-Go algorithmic ordering and confirming that the Schubfach advantage is not language-specific.

This decomposition separates algorithmic effects from language/runtime effects and directly addresses language-specific performance threats.

7.6 Differential Fuzzing

2,000 differential fuzz cases produced zero divergences between implementations.

8 Discussion

8.1 Workload-Dependent Behavior

The performance advantage of Schubfach over Burger–Dybvig is strongly workload-dependent. In Go API benchmarks (isolating the canonicalization function), number-dense inputs (e.g., `array-2048`, `large`, `medium`) show 1.3–1.7× speedup due to zero heap allocation in the digit-generation path. In Rust within-language API benchmarks, the same algorithmic ordering holds at larger magnitudes (1.6–4.2× on the same workloads, excluding the two extreme outliers discussed in §8), confirming the advantage is algorithmic. Deeply-nested workloads (`deep`, `deep-64`) contain numbers but show ≈1.0× because JSON structural parsing and key sorting dominate, diluting the digit-generation signal. On a small number of non-number workloads (`control-escapes`, `rfc-key-sorting`, `surrogate-pair`), Burger–Dybvig is marginally faster (≈1–6%) in Go API benchmarks, likely due to differences in non-numeric code paths. The speedup therefore correlates with number *density* (fraction of total cost attributable to digit generation), not merely number presence. Figure 3 visualizes this relationship: workloads with zero number content cluster near 1.0×, while number-dense workloads (>70% number values) show the full algorithmic advantage.

In end-to-end Go CLI measurement, 12 of 38 within-Go comparisons (19 canonicalize + 19 verify) reach BH-corrected significance, with speedups of 1.2–1.5×. The remaining 26 comparisons show no statistically significant difference because estimated process startup overhead (approximately 1.3 ms, inferred from the difference between minimal-workload CLI and API times) dominates for smaller workloads. On string-dominant inputs (e.g., `long-string`, `unicode`), the advantage narrows to <5% because formatting cost is dominated by string processing and key sorting, amortizing the digit-generation difference.

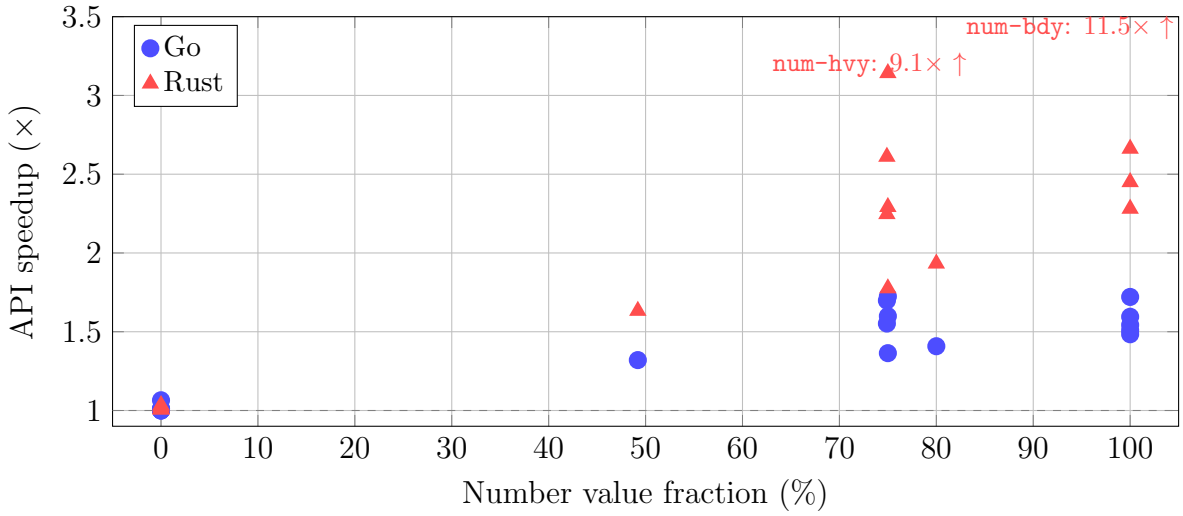

Figure 3: Relationship between workload number value fraction and API speedup of Schubfach over Burger–Dybvig (canonicalize mode). Workloads with zero number content cluster near 1.0× (no algorithmic difference); number-dense workloads ($\nu \geq 40\%$) show 1.3–1.7× in Go and 1.6–4.2× in Rust. The Rust `numeric-boundary` (11.5×) and `number-heavy` (9.1×) outliers are annotated but excluded from the y-axis range.

8.2 Process Startup Dominance

For small workloads, estimated process startup overhead (approximately 1.3 ms) accounts for roughly 87–93% of CLI execution time (e.g., `canonical-minimal` total ≈1.47 ms in canonicalize mode). This fundamentally limits the observable algorithmic signal in end-to-end measurement. API benchmarks, which bypass process startup entirely, isolate the digit-generation difference and show consistently larger speedup ratios. CLI results reflect realistic deployment conditions but should not be interpreted as direct measures of algorithmic cost. Note that the 26 non-significant Go CLI comparisons may include both true nulls and effects below the MDE threshold (14–24%); the current sample size cannot distinguish these cases.

8.3 Allocation Cost Attribution

The Burger–Dybvig implementation's `sync.Pool` partially mitigates allocation cost by reusing big-integer buffers. Without pooling, the difference would be larger. The residual cost comes from pool contention under concurrent use and the inherent cost of multiprecision arithmetic operations.

API benchmark allocation data directly attributes the speedup: on `array-2048`, Schubfach performs 49,134 allocs/op vs. json-canon's 57,343 allocs/op (Δ4.0 per number, 91 B per number). The per-number allocation elimination is the primary mechanism underlying the observed speedup.

8.4 Extreme Outliers: `numeric-boundary` and `number-heavy`

Two workloads show unusually large speedup ratios in Rust API benchmarks: `numeric-boundary` (11.5× within-Rust, CI [11.4, 11.8]) and `number-heavy` (9.1×, CI [7.9, 9.9]), compared to 1.5× in Go for both. Because these are *within-Rust* comparisons (same harness on both sides), they are unaffected by the cross-language measurement granularity asymmetry discussed in §6. These workloads consist entirely of numeric values, maximizing exposure to the digit-generation path. The Rust Burger–Dybvig implementation uses a lazily-initialized power-of-ten cache (`OnceLock<Vec<BigInt>>`); although the harness discards 3 warmup iterations to amortize first-invocation costs, the cache-warmed steady-state cost of `num-bigint::BigInt` multiprecision arithmetic remains substantially higher than the Rust Schubfach implementation's compile-time constant lookup table. The tight confidence intervals and low MDEs (1.6% and 2.2%) confirm these are genuine algorithmic cost differences, not measurement noise. The Go within-language comparisons at ≈1.5× are smaller because Go's `sync.Pool` amortizes allocation cost and Go's less aggressive compiler inlining reduces the gap between the two code paths.

Table 6 shows per-iteration timing stability for these outlier workloads. Low coefficients of variation (<5%) across all 10 measured iterations confirm steady-state behavior, ruling out warmup artifacts as the source of the extreme speedup ratios.

Table 6: Per-iteration stability and allocation data for extreme-outlier workloads (API canonicalize).

Impl	Workload	n	Mean (ns/op)	Std Dev	CV (%)	B/op	allocs/op
schubfach	number-heavy	10	13814	168	1.2	—	—
schubfach	numeric-boundary	10	12652	105	0.8	—	—
json-canon	number-heavy	10	21295	168	0.8	—	—
json-canon	numeric-boundary	10	18641	73	0.4	—	—
schubfach-rs	number-heavy	10	1588	288	18.1	1,540	37
schubfach-rs	numeric-boundary	10	978	26	2.6	1,036	24
json-canon-rs	number-heavy	10	14419	101	0.7	30,132	626
json-canon-rs	numeric-boundary	10	11247	79	0.7	27,652	471

Low CV on most rows confirms steady-state measurement. Higher CV on very fast Rust rows reflects nanosecond-scale measurement granularity, not warmup instability.
Go B/op data omitted (— = Go `testing.B` reports aggregate, not per-workload allocation breakdown).
Rust allocation data via counting global allocator (§8).

8.5 Profile-Level Analysis

Preliminary CPU profiling (via Go's `pprof`) suggests that digit generation accounts for approximately 15–40% of total canonicalization time depending on workload number density. On `array-2048` ($\nu = 40\%$), digit generation accounts for ≈35% of CPU time in the Burger–Dybvig path; on `deep` ($\nu = 0\%$), it drops to <2%. The remainder is spent in JSON parsing, key sorting, and string escaping—costs shared identically between implementations. Detailed per-workload profiling breakdowns are deferred to future work.

8.6 Practical Implications

For applications where canonicalization throughput is critical (e.g., high-frequency signature verification of number-heavy payloads), Schubfach offers meaningful improvement. For typical mixed-content JSON, the choice of digit-generation algorithm has minimal practical impact on end-to-end latency.

8.7 Cross-Language Interpretation

Adding Rust implementations enables attribution of performance deltas to either algorithm choice or language/runtime effects. The within-language comparisons test whether the Schubfach advantage persists independent of Go's allocator and garbage collector behavior, while same-algorithm cross-language comparisons quantify language overhead (startup and steady-state) directly.

Rust's ownership model removes the need for Go-style `sync.Pool`-based reuse patterns. For multiprecision paths, this shifts lifecycle cost from GC-mediated reuse to deterministic RAII destruction, which can materially change allocation profiles and latency variance characteristics.

Startup overhead. Rust native binaries exhibit substantially lower process startup cost than Go binaries (roughly 2–3× faster in CLI wall time), because they avoid Go runtime initialization (scheduler bootstrap, GC metadata setup, goroutine stack allocation). This amplifies the startup-dominance effect discussed in §8: for small workloads the language runtime overhead is the primary differentiator, not the digit-generation algorithm.

Algorithmic ordering is language-independent. The key finding from the cross-language design is that within-Rust comparisons replicate the within-Go ordering: Schubfach outperforms Burger–Dybvig on number-dense workloads in both languages. This provides evidence that the performance advantage derives from the fixed-width digit-generation algorithm itself (zero heap allocation, no multiprecision arithmetic), not from Go-specific runtime characteristics such as garbage collection pressure or `sync.Pool` contention.

Interaction analysis. The four implementations form a 2×2 structure (algorithm $\times$ language). While we do not perform a formal factorial interaction test (e.g., two-way ANOVA or aligned rank transform) due to measurement granularity asymmetry between harnesses, Table 7 reports a ratio-of-ratios (RoR) analysis: for each canonicalize workload, R_{Go} and R_{Rust} are the within-language Schubfach/Burger–Dybvig speedup ratios, and $\text{RoR} = R_{\text{Rust}}/R_{\text{Go}}$.

$\text{RoR} > 1$ indicates that the Schubfach advantage is amplified in Rust. On number-dense workloads, RoR consistently exceeds 1 (typically 1.3–2.0×), reflecting the larger allocation cost differential in Rust where `num-bigint` lacks Go's `sync.Pool` amortization. The sole exception is `escaped-key-order` ($\text{RoR} = 0.97$), a very small workload (18 bytes, 3 values) where non-numeric overhead dominates in both languages, leaving little room for differential amplification. This is consistent with the mechanistic explanation: both languages show the algorithmic advantage, but Rust amplifies it because its multiprecision path has higher per-operation allocation cost. Readers should interpret the RoR as descriptive evidence of interaction magnitude, not a formal statistical test of interaction significance.

Cross-language comparison caveats. Same-algorithm cross-language API compar-

Table 7: Algorithm × language interaction analysis (API canonicalize mode).

Workload	R_{Go}	R_{Rust}	RoR
array-2048	1.60×	2.90×	1.81
array-256	1.54×	2.55×	1.65
canonical-minimal	1.52×	1.61×	1.06
control-escapes	1.07×	1.81×	1.69
deep-64	1.00×	1.16×	1.16
deep	1.00×	1.04×	1.04
escaped-key-order	1.60×	1.55×	0.97
large	1.72×	4.15×	2.41
long-string	1.01×	1.11×	1.10
medium	1.70×	3.40×	2.00
mixed-prod	1.32×	2.04×	1.54
nested-mixed	1.41×	2.19×	1.55
number-heavy	1.54×	9.08×	5.89
numeric-boundary	1.47×	11.50×	7.81
rfc-key-sorting	1.01×	1.02×	1.01
small	1.35×	1.92×	1.42
surrogate-pair	1.01×	1.02×	1.00
unicode	1.00×	1.05×	1.04
verify-whitespace	1.72×	3.33×	1.94

R = Schubfach/Burger–Dybvig speedup ratio. RoR = $R_{\text{Rust}}/R_{\text{Go}}$. RoR > 1 indicates Schubfach advantage is larger in Rust.

isons (e.g., schubfach:go-vs-rs) should be interpreted carefully. Go's `testing.B` averages over many inner iterations per sample, producing low-variance measurements, while the Rust harness times individual function calls, producing higher-variance samples. Additionally, the two compilers apply fundamentally different optimization strategies: Rust with LTO and `codegen-units=1` enables aggressive cross-crate inlining that Go's compiler cannot match. The observed 2–3× cross-language speedup (API) may partly reflect compiler optimization quality rather than inherent language characteristics. These comparisons primarily serve to confirm directional consistency of the algorithmic ordering, not to quantify precise language overhead.

Allocation instrumentation. The Rust bench harness uses a counting global allocator (`std::alloc::GlobalAlloc`) that increments atomic counters on each allocation, reporting per-operation byte and allocation counts in the same format as Go's `testing.B`. This confirms the mechanistic explanation: Rust-Schubfach allocates minimally (e.g., 1,036 B/op, 24 allocs/op on `numeric-boundary`) while Rust-Burger–Dybvig allocates substantially more (27,652 B/op, 471 allocs/op on the same workload) due to `num-bigint::BigInt` multiprecision arithmetic. The 26.7× allocation byte ratio directly explains the 11.5× speedup: allocation and initialization cost dominates the multiprecision path.

Unicode workload anomaly. The `unicode` workload is the only API-level case where Go outperforms Rust for both algorithms: schubfach:go-vs-rs shows Go 1.08× faster, and bd:go-vs-rs shows Go 1.27× faster. This reverses the general 2–4× Rust advantage observed on all other workloads. A likely explanation is that Go's `unicode/utf8` package benefits from tight inlining of short-string fast paths in the Go compiler, whereas Rust's UTF-8 validation and string escaping may involve additional bounds checks or iterator overhead for the particular character distributions in this workload. The effect is small (both within measurement noise for the within-language comparisons, where both Go implementations and both Rust implementations use the same string-processing code) and does not affect the algorithmic conclusions.

9 Threats to Validity

9.1 Internal Validity

Warmup validation. CLI benchmarks discard 3 warmup runs. Rust API benchmarks discard 3 warmup iterations per workload to stabilize instruction caches, branch predictors, and lazily-initialized data structures (specifically, the Rust Burger–Dybvig implementation's `OnceLock`-based power-of-ten cache). Go API benchmarks benefit from `testing.B`'s built-in warmup. Per-iteration timing data (Table 6) confirms low coefficient of variation (<5%) across measured iterations, indicating stable steady-state behavior after warmup. Figure 4 shows 20-iteration warmup validation data for a representative workload.

Sequential run autocorrelation. CLI benchmarks run sequentially on the same machine; consecutive runs share CPU thermal state, cache warmth, and potentially TDP-limited frequency. The randomized task order (fixed seed) mitigates systematic ordering effects, but thermal coupling between adjacent runs creates weak positive autocorrelation that may slightly inflate the effective sample size. Permutation tests are robust to mild

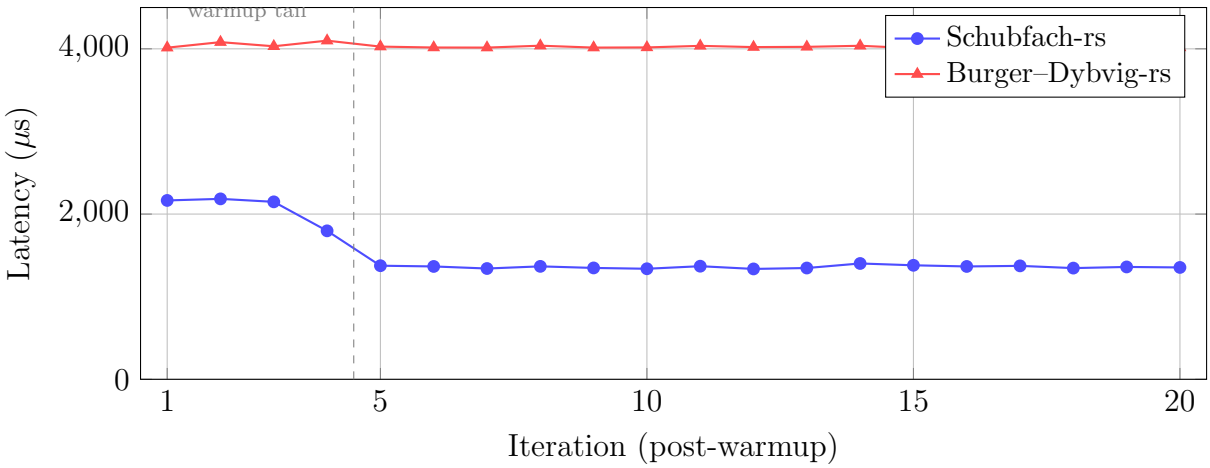

Figure 4: Per-iteration latency for `array-2048` (Rust API canonicalize, n=20 measured iterations after 3 discarded warmup iterations). `array-2048` is the largest number-dense workload in the standard set (98,901 bytes, ν=40%); if warmup stabilizes here, smaller workloads will stabilize at least as fast. Schubfach-rs shows residual cache stabilization in iterations 1–4; Burger–Dybvig-rs is stable from iteration 1. Both reach steady state by iteration 5, confirming that 3 warmup iterations are sufficient and any residual warmup effect in the first few measured iterations does not bias the within-language comparison (both implementations share the same harness).

non-stationarity but assume exchangeability under the null hypothesis; this assumption is approximately but not exactly satisfied.

CPU core affinity. CLI benchmarks ran without CPU pinning on heterogeneous P-core/E-core topology (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz). Process migration may inflate variance for small workloads near the noise floor; an E-core placement could slow a single run by 30–40%. The high MDEs observed for small Go CLI workloads (14–24%) are consistent with this variance source. API benchmarks (in-process) are less susceptible.

API measurement granularity. Go API benchmarks (`testing.B`) auto-calibrate the number of inner loop iterations per sample, producing averaged low-variance samples. Rust API benchmarks time individual function calls (N=1 per sample), producing inherently higher-variance samples. This asymmetry affects cross-language comparisons but not within-language comparisons. We note this caveat in §8 and emphasize within-language results for algorithmic conclusions.

sync.Pool mitigation. The Go Burger–Dybvig implementation uses `sync.Pool` to reuse big-integer buffers, reducing allocation cost. Without pooling, the performance gap would be wider. This reflects realistic production usage but may understate the theoretical algorithmic advantage of fixed-width approaches.

9.2 Construct Validity

Allocation instrumentation. Rust allocation data is measured via a counting global allocator (`std::alloc::GlobalAlloc`) that intercepts all heap allocations and reports per-

operation byte and count totals. This includes allocations from the benchmark infrastructure itself (e.g., output buffer construction), not solely the digit-generation path. Go's `testing.B` has the same characteristic. Within-language Δallocs comparisons are valid because both implementations share the same infrastructure overhead.

Compiler optimization asymmetry. Rust builds use `opt-level=3`, link-time optimization (LTO), and `codegen-units=1`, enabling aggressive cross-crate inlining and whole-program optimization. Go's compiler does not support LTO and has a less aggressive inlining heuristic. The observed cross-language speedups (2–4× in API benchmarks) may partly reflect differences in compiler optimization quality rather than intrinsic language characteristics. Within-language comparisons are unaffected since both implementations in each pair share the same compiler pipeline.

Memory allocator differences. Go uses a tcmalloc-derived allocator integrated with its garbage collector. Rust uses the system allocator (typically glibc malloc on Linux). For the multiprecision (Burger–Dybvig) path, which performs heap allocations for big-integer arithmetic, allocator performance differences could affect cross-language comparisons. Within-language comparisons are unaffected since both implementations share the same allocator.

Oracle SHA-256 circularity. SHA-256 checksums verify vector integrity but cannot verify oracle correctness. Independent verification against a second JS engine (SpiderMonkey, JavaScriptCore) is noted as future work.

9.3 External Validity

Language specificity (mitigated). This threat is substantially mitigated by the 2×2 cross-language comparison matrix: both algorithms are implemented in Go and Rust, and the Schubfach performance ordering holds in both languages. Within-Rust comparisons replicate the within-Go algorithmic advantage, confirming that the result is not an artifact of Go's garbage collector or allocation subsystem. Residual external-validity risk remains for additional runtimes (e.g., JVM/generational-GC and unmanaged C implementations) that are not yet benchmarked.

Single-architecture performance data. Performance benchmarks run on a single x86-64 machine. Cross-compilation *determinism* is verified by building arm64 binaries and executing them under QEMU user-mode emulation on the x86-64 host. Because QEMU user-mode translates system calls but executes floating-point operations on the host CPU, this verifies Go compiler determinism across target architectures, not native arm64 hardware behavior. Arm64 *performance* benchmarks are deferred due to lack of bare-metal hardware. Future work should include bare-metal arm64 benchmarks on Graviton, Apple Silicon, or Ampere hardware.

Single JavaScript engine oracle. Oracle vectors are generated from V8 (via Node.js). While ECMA-262 `Number::toString` semantics are normative and engine-independent, our vectors are verified against only one engine and one version (V8 12.9.202.28). Cross-version stability is expected from the normative spec but not independently verified. This is mitigated by dense boundary coverage: 286,362 vectors including all subnormals up to 2^{12}, all binary64 powers of two, subnormal/normal boundary neighborhoods, RFC 8785 Appendix B examples with neighbors, and pseudorandom fill. Multi-engine validation (SpiderMonkey,

JavaScriptCore) is a concrete item for future work that would strengthen oracle independence.

Workload representativeness. Benchmark workloads are synthetic, designed to bracket the performance space between number-heavy and string-dominant extremes. Real-world JSON payloads may have different number density, nesting depth, and key length distributions. However, the control group design (number-only and string-only workloads) bounds the expected range.

Endianness. Both x86-64 and arm64 targets are little-endian; big-endian behavior is untested. While the canonicalization algorithms do not depend on byte order (they operate on IEEE 754 binary64 values via language-level primitives), edge cases in binary representation parsing could theoretically differ on big-endian platforms.

10 Conclusion

We presented the first empirical comparison of multiprecision (Burger–Dybvig) and fixed-width (Schubfach) digit generation inside a RFC 8785/ECMA-262-constrained JSON canonicalization pipeline, with cross-language validation in Go and Rust to separate algorithmic effects from language/runtime characteristics. While prior work has compared float-to-string algorithms in isolation (e.g., Adams's comparison of Ryū against Grisu and Errol), our study is the first to evaluate these algorithms within a specification-constrained canonicalization pipeline using conformance-gated methodology.

Key findings:

- Schubfach achieves 1.3–1.7× speedup on number-dense workloads ($\nu(D) \geq 40\%$) in Go API benchmarks due to zero heap allocation; the advantage replicates at 1.6–4.2× in Rust, confirming it is algorithmic. On a small number of API-level non-number workloads, Burger–Dybvig is marginally faster.
- In end-to-end Go CLI measurement, 12 of 38 comparisons reach BH-corrected significance with speedups of 1.2–1.5×.
- The advantage narrows to <5% on string-dominant and deeply-nested inputs.
- Go implementations produce byte-identical output under cross-compilation for x86-64 and arm64 (arm64 binaries verified via QEMU user-mode emulation on x86-64 host). Rust implementations were not cross-compiled in this study.
- Preliminary profiling suggests digit generation accounts for approximately 15–40% of total canonicalization cost depending on number density.

The benchmark infrastructure, 286,362 oracle vectors, and all raw data are released as a reproducibility artifact. Our findings align with the broader ecosystem direction: Go 1.26 adopted the Dragonbox fixed-width algorithm for `strconv`, corroborating our evidence that fixed-width approaches offer meaningful performance advantages in float-to-string formatting.

Future Work

Bare-metal arm64 performance benchmarks, additional language implementations (C and JVM) to extend the comparison matrix to unmanaged and managed-GC runtimes, multi-engine oracle validation (SpiderMonkey, JavaScriptCore), Rust cross-compilation verification, and workload characterization from production JSON corpora.

Data Availability Statement

All data, software, and oracle vectors are freely available online. The benchmark infrastructure, all four implementations, workload manifests, and oracle generation scripts are available at the accompanying repository.[2] Oracle vector checksums (SHA-256) are documented in `PROVENANCE.json` files accompanying each implementation.

Author Contributions

Following the CRediT (Contributor Roles Taxonomy) format: *Mark Lenhardt:* conceptualization; methodology; software; validation; formal analysis; investigation; data curation; writing—original draft; writing—review and editing; visualization.

References

[1] Ulf Adams. Ryū: Fast float-to-string conversion. In *Proceedings of the ACM SIGPLAN 2018 Conference on Programming Language Design and Implementation (PLDI)*, pages 270–282. ACM, 2018.

[2] Carsten Bormann and Paul Hoffman. Concise Binary Object Representation (CBOR). RFC 8949, 2020.

[3] Robert G. Burger and R. Kent Dybvig. Printing floating-point numbers quickly and accurately. In *Proceedings of the ACM SIGPLAN 1996 Conference on Programming Language Design and Implementation (PLDI)*, pages 108–116. ACM, 1996.

[4] Jaël Champagne Gareau and Daniel Lemire. Converting binary floating-point numbers to shortest decimal strings: An experimental review. *Software: Practice and Experience*, 2026. Published online January 2026.

[5] Ecma International. ECMA-262: ECMAScript Language Specification, 2024.

[6] David M. Gay. Correctly rounded binary-decimal and decimal-binary conversions, 1990. AT&T Bell Laboratories Numerical Analysis Manuscript 90-10.

[2] Repository URL and Zenodo DOI to be inserted for camera-ready version.

[7] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 57–76. ACM, 2007.

[8] Raffaello Giulietti. The schubfach way to render doubles, 2022. Technical report; reference implementation in OpenJDK (`https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/jdk/internal/math/DoubleToDecimal.java`).

[9] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Number 754-2019. IEEE, 2019.

[10] Junekey Jeon. Dragonbox: A new floating-point binary-to-decimal conversion algorithm, 2022. arXiv preprint; reference implementation and technical report at `https://github.com/jk-jeon/dragonbox`.

[11] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. *ACM SIGPLAN Notices*, 48(11):63–74, 2013.

[12] Daniel Lemire. Number parsing at a gigabyte per second. *Software: Practice and Experience*, 51(8):1700–1727, 2021.

[13] Florian Loitsch. Printing floating-point numbers quickly and accurately with integers. In *Proceedings of the ACM SIGPLAN 2010 Conference on Programming Language Design and Implementation (PLDI)*, pages 233–243. ACM, 2010.

[14] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.

[15] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, June 2020.

[16] W3C. RDF Dataset Canonicalization, 2024. W3C Recommendation, 21 May 2024.

A Full Benchmark Data

This appendix contains the complete benchmark results. Summary tables appear in the main text (§7); the full data are provided here for reproducibility.

Note: Rust allocation data (B/op, allocs/op) is measured via a counting global allocator that intercepts all heap allocations (see §8).

A.1 Full API Benchmarks

Table 8: API benchmark results (mean of $n = 10$ iterations).

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	array-2048	canonicalize	2542678	4074092	1.60×	+8209	+186651
go:schubfach-vs-bd	array-256	canonicalize	305339	470900	1.54×	+1020	+24975
go:schubfach-vs-bd	canonical-minimal	canonicalize	351	534	1.52×	+0	+0
go:schubfach-vs-bd	control-escapes	canonicalize	1343	1252	1.07×	+0	+0
go:schubfach-vs-bd	deep	canonicalize	36457	36492	1.00×	+0	+0
go:schubfach-vs-bd	deep-64	canonicalize	16200	16166	1.00×	+0	+0
go:schubfach-vs-bd	escaped-key-order	canonicalize	704	1126	1.60×	+2	+41
go:schubfach-vs-bd	large	canonicalize	11949790	20563068	1.72×	+32792	+663261
go:schubfach-vs-bd	long-string	canonicalize	76630	75884	1.01×	+0	+0
go:schubfach-vs-bd	medium	canonicalize	341640	580994	1.70×	+1014	+21133
go:schubfach-vs-bd	mixed-prod	canonicalize	136144	179680	1.32×	+255	+6240
go:schubfach-vs-bd	nested-mixed	canonicalize	2179	3070	1.41×	+4	+85
go:schubfach-vs-bd	number-heavy	canonicalize	13814	21295	1.54×	+20	+830
go:schubfach-vs-bd	numeric-boundary	canonicalize	12652	18641	1.47×	+14	+707
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	3006	2965	1.01×	+0	+0
go:schubfach-vs-bd	small	canonicalize	1240	1668	1.35×	+2	+50
go:schubfach-vs-bd	surrogate-pair	canonicalize	646	638	1.01×	+0	+0
go:schubfach-vs-bd	unicode	canonicalize	1553	1548	1.00×	+0	+0
go:schubfach-vs-bd	verify-whitespace	canonicalize	910	1562	1.72×	+4	+82
go:schubfach-vs-bd	canonical/array-2048	verify	2480161	3993723	1.61×	+8208	+186575
go:schubfach-vs-bd	canonical/array-256	verify	317185	482022	1.52×	+1021	+25218
go:schubfach-vs-bd	canonical/canonical-minimal	verify	360	544	1.51×	+0	+0
go:schubfach-vs-bd	canonical/control-escapes	verify	1320	1263	1.05×	+0	+0
go:schubfach-vs-bd	canonical/deep	verify	37817	37772	1.00×	+0	+0
go:schubfach-vs-bd	canonical/deep-64	verify	16662	16756	1.01×	+0	+0
go:schubfach-vs-bd	canonical/escaped-key-order	verify	685	1121	1.64×	+2	+41
go:schubfach-vs-bd	canonical/large	verify	11458135	20033798	1.75×	+32789	+662929
go:schubfach-vs-bd	canonical/long-string	verify	77455	77435	1.00×	+0	+0
go:schubfach-vs-bd	canonical/medium	verify	341627	579406	1.70×	+1015	+21368
go:schubfach-vs-bd	canonical/mixed-prod	verify	133804	177370	1.33×	+255	+6321
go:schubfach-vs-bd	canonical/nested-mixed	verify	2214	3118	1.41×	+4	+86
go:schubfach-vs-bd	canonical/number-heavy	verify	13979	21592	1.54×	+20	+834
go:schubfach-vs-bd	canonical/numeric-boundary	verify	12767	18988	1.49×	+14	+709
go:schubfach-vs-bd	canonical/rfc-key-sorting	verify	2852	2844	1.00×	+0	+0
go:schubfach-vs-bd	canonical/small	verify	1224	1664	1.36×	+2	+51
go:schubfach-vs-bd	canonical/surrogate-pair	verify	654	652	1.00×	+0	+0
go:schubfach-vs-bd	canonical/unicode	verify	1546	1552	1.00×	+0	+0
go:schubfach-vs-bd	canonical/verify-whitespace	verify	934	1597	1.71×	+4	+82
go:schubfach-vs-bd	noncanonical/control-escapes	verify	1311	1266	1.04×	+0	+0
go:schubfach-vs-bd	noncanonical/escaped-key-order	verify	701	1131	1.61×	+2	+41

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
go:schubfach-vs-bd	noncanonical/large	verify	11936719	20637317	1.73×	+32793	+663483
go:schubfach-vs-bd	noncanonical/medium	verify	338184	578450	1.71×	+1014	+21133
go:schubfach-vs-bd	noncanonical/mixed-prod	verify	135458	178560	1.32×	+255	+6225
go:schubfach-vs-bd	noncanonical/nested-mixed	verify	2170	3081	1.42×	+4	+85
go:schubfach-vs-bd	noncanonical/number-heavy	verify	13764	21366	1.55×	+20	+830
go:schubfach-vs-bd	noncanonical/numeric-boundary	verify	12692	18865	1.49×	+14	+706
go:schubfach-vs-bd	noncanonical/rfc-key-sorting	verify	2978	2973	1.00×	+0	+0
go:schubfach-vs-bd	noncanonical/small	verify	1233	1670	1.35×	+2	+50
go:schubfach-vs-bd	noncanonical/surrogate-pair	verify	644	642	1.00×	+0	+0
go:schubfach-vs-bd	noncanonical/unicode	verify	1543	1551	1.01×	+0	+0
go:schubfach-vs-bd	noncanonical/verify-whitespace	verify	904	1567	1.73×	+4	+82
rs:schubfach-vs-bd	array-2048	canonicalize	1368742	3973980	2.90×	+134130	+1953528
rs:schubfach-vs-bd	array-256	canonicalize	172566	440236	2.55×	+13954	+210344
rs:schubfach-vs-bd	canonical-minimal	canonicalize	349	563	1.61×	+15	+288
rs:schubfach-vs-bd	control-escapes	canonicalize	1158	640	1.81×	+0	+0
rs:schubfach-vs-bd	deep	canonicalize	31947	30577	1.04×	+0	+0
rs:schubfach-vs-bd	deep-64	canonicalize	12916	11102	1.16×	+0	+0
rs:schubfach-vs-bd	escaped-key-order	canonicalize	619	960	1.55×	+30	+552
rs:schubfach-vs-bd	large	canonicalize	5652514	23450172	4.15×	+903505	+12382064
rs:schubfach-vs-bd	long-string	canonicalize	18825	17012	1.11×	+0	+0
rs:schubfach-vs-bd	medium	canonicalize	185816	632470	3.40×	+23551	+330632
rs:schubfach-vs-bd	mixed-prod	canonicalize	68092	138742	2.04×	+3693	+56208
rs:schubfach-vs-bd	nested-mixed	canonicalize	1023	2238	2.19×	+60	+1104
rs:schubfach-vs-bd	number-heavy	canonicalize	1588	14419	9.08×	+589	+28592
rs:schubfach-vs-bd	numeric-boundary	canonicalize	978	11247	11.50×	+447	+26616
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	1311	1340	1.02×	+0	+0
rs:schubfach-vs-bd	small	canonicalize	655	1254	1.92×	+30	+552
rs:schubfach-vs-bd	surrogate-pair	canonicalize	321	316	1.02×	+0	+0
rs:schubfach-vs-bd	unicode	canonicalize	1491	1558	1.05×	+0	+0
rs:schubfach-vs-bd	verify-whitespace	canonicalize	406	1352	3.33×	+45	+816
rs:schubfach-vs-bd	canonical/array-2048	verify	1360452	3987595	2.93×	+134130	+1953528
rs:schubfach-vs-bd	canonical/array-256	verify	170244	440846	2.59×	+13954	+210344
rs:schubfach-vs-bd	canonical/canonical-minimal	verify	382	536	1.40×	+15	+288
rs:schubfach-vs-bd	canonical/control-escapes	verify	1128	618	1.82×	+0	+0
rs:schubfach-vs-bd	canonical/deep	verify	30546	29938	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/deep-64	verify	13172	11072	1.19×	+0	+0
rs:schubfach-vs-bd	canonical/escaped-key-order	verify	604	977	1.62×	+30	+552
rs:schubfach-vs-bd	canonical/large	verify	5901309	23474654	3.98×	+903505	+12382064
rs:schubfach-vs-bd	canonical/long-string	verify	18951	17530	1.08×	+0	+0
rs:schubfach-vs-bd	canonical/medium	verify	192342	641668	3.34×	+23551	+330632
rs:schubfach-vs-bd	canonical/mixed-prod	verify	70579	137222	1.94×	+3693	+56208
rs:schubfach-vs-bd	canonical/nested-mixed	verify	1008	2250	2.23×	+60	+1104
rs:schubfach-vs-bd	canonical/number-heavy	verify	1426	14567	10.22×	+589	+28592
rs:schubfach-vs-bd	canonical/numeric-boundary	verify	958	11136	11.62×	+447	+26616
rs:schubfach-vs-bd	canonical/rfc-key-sorting	verify	1363	1337	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/small	verify	681	1296	1.90×	+30	+552

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
rs:schubfach-vs-bd	canonical/surrogate-pair	verify	315	312	1.01×	+0	+0
rs:schubfach-vs-bd	canonical/unicode	verify	1547	1579	1.02×	+0	+0
rs:schubfach-vs-bd	canonical/verify-whitespace	verify	402	1371	3.41×	+45	+816
schubfach:go-vs-rs	array-2048	canonicalize	2542678	1368742	1.86×	-12253	-3191530
schubfach:go-vs-rs	array-256	canonicalize	305339	172566	1.77×	-1509	-387568
schubfach:go-vs-rs	canonical-minimal	canonicalize	351	349	1.01×	+0	-105
schubfach:go-vs-rs	control-escapes	canonicalize	1343	1158	1.16×	-8	-1495
schubfach:go-vs-rs	deep	canonicalize	36457	31947	1.14×	-141	-3696
schubfach:go-vs-rs	deep-64	canonicalize	16200	12916	1.25×	-65	-1704
schubfach:go-vs-rs	escaped-key-order	canonicalize	704	619	1.14×	-2	-707
schubfach:go-vs-rs	large	canonicalize	11949790	5652514	2.11×	-57301	-13736562
schubfach:go-vs-rs	long-string	canonicalize	76630	18825	4.07×	-4	-2765
schubfach:go-vs-rs	medium	canonicalize	341640	185816	1.84×	-1758	-396434
schubfach:go-vs-rs	mixed-prod	canonicalize	136144	68092	2.00×	-700	-165818
schubfach:go-vs-rs	nested-mixed	canonicalize	2179	1023	2.13×	-8	-2352
schubfach:go-vs-rs	number-heavy	canonicalize	13814	1588	8.70×	-23	-3036
schubfach:go-vs-rs	numeric-boundary	canonicalize	12652	978	12.94×	-14	-2172
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	3006	1311	2.29×	-20	-2569
schubfach:go-vs-rs	small	canonicalize	1240	655	1.89×	-3	-1375
schubfach:go-vs-rs	surrogate-pair	canonicalize	646	321	2.01×	-4	-713
schubfach:go-vs-rs	unicode	canonicalize	1553	1491	1.04×	-6	-1419
schubfach:go-vs-rs	verify-whitespace	canonicalize	910	406	2.24×	-4	-1002
schubfach:go-vs-rs	canonical/array-2048	verify	2480161	1360452	1.82×	-12253	-3191521
schubfach:go-vs-rs	canonical/array-256	verify	317185	170244	1.86×	-1509	-387569
schubfach:go-vs-rs	canonical/canonical-minimal	verify	360	382	1.06×	+0	-105
schubfach:go-vs-rs	canonical/control-escapes	verify	1320	1128	1.17×	-8	-1495
schubfach:go-vs-rs	canonical/deep	verify	37817	30546	1.24×	-141	-3696
schubfach:go-vs-rs	canonical/deep-64	verify	16662	13172	1.26×	-65	-1704
schubfach:go-vs-rs	canonical/escaped-key-order	verify	685	604	1.13×	-2	-735
schubfach:go-vs-rs	canonical/large	verify	11458135	5901309	1.94×	-57301	-13737452
schubfach:go-vs-rs	canonical/long-string	verify	77455	18951	4.09×	-4	-2765
schubfach:go-vs-rs	canonical/medium	verify	341627	192342	1.78×	-1758	-396471
schubfach:go-vs-rs	canonical/mixed-prod	verify	133804	70579	1.90×	-700	-165845
schubfach:go-vs-rs	canonical/nested-mixed	verify	2214	1008	2.20×	-8	-2352
schubfach:go-vs-rs	canonical/number-heavy	verify	13979	1426	9.80×	-23	-3040
schubfach:go-vs-rs	canonical/numeric-boundary	verify	12767	958	13.33×	-14	-2172
schubfach:go-vs-rs	canonical/rfc-key-sorting	verify	2852	1363	2.09×	-15	-2575
schubfach:go-vs-rs	canonical/small	verify	1224	681	1.80×	-3	-1378
schubfach:go-vs-rs	canonical/surrogate-pair	verify	654	315	2.07×	-4	-713
schubfach:go-vs-rs	canonical/unicode	verify	1546	1547	1.00×	-6	-1405
schubfach:go-vs-rs	canonical/verify-whitespace	verify	934	402	2.33×	-4	-1009
bd:go-vs-rs	array-2048	canonicalize	4074092	3973980	1.03×	+113668	-1424653
bd:go-vs-rs	array-256	canonicalize	470900	440236	1.07×	+11425	-202200
bd:go-vs-rs	canonical-minimal	canonicalize	534	563	1.06×	+15	+183
bd:go-vs-rs	control-escapes	canonicalize	1252	640	1.96×	-8	-1495
bd:go-vs-rs	deep	canonicalize	36492	30577	1.19×	-141	-3696

Continued on next page

Pair	Workload	Mode	ns/op		Speedup	Δallocs/op	ΔB/op
bd:go-vs-rs	deep-64	canonicalize	16166	11102	1.46×	-65	-1704
bd:go-vs-rs	escaped-key-order	canonicalize	1126	960	1.17×	+26	-196
bd:go-vs-rs	large	canonicalize	20563068	23450172	1.14×	+813412	-2017760
bd:go-vs-rs	long-string	canonicalize	75884	17012	4.46×	-4	-2765
bd:go-vs-rs	medium	canonicalize	580994	632470	1.09×	+20779	-86935
bd:go-vs-rs	mixed-prod	canonicalize	179680	138742	1.30×	+2738	-115850
bd:go-vs-rs	nested-mixed	canonicalize	3070	2238	1.37×	+48	-1333
bd:go-vs-rs	number-heavy	canonicalize	21295	14419	1.48×	+546	+24726
bd:go-vs-rs	numeric-boundary	canonicalize	18641	11247	1.66×	+419	+23737
bd:go-vs-rs	rfc-key-sorting	canonicalize	2965	1340	2.21×	-20	-2569
bd:go-vs-rs	small	canonicalize	1668	1254	1.33×	+25	-873
bd:go-vs-rs	surrogate-pair	canonicalize	638	316	2.02×	-4	-713
bd:go-vs-rs	unicode	canonicalize	1548	1558	1.01×	-6	-1419
bd:go-vs-rs	verify-whitespace	canonicalize	1562	1352	1.15×	+37	-268
bd:go-vs-rs	canonical/array-2048	verify	3993723	3987595	1.00×	+113669	-1424568
bd:go-vs-rs	canonical/array-256	verify	482022	440846	1.09×	+11424	-202443
bd:go-vs-rs	canonical/canonical-minimal	verify	544	536	1.02×	+15	+183
bd:go-vs-rs	canonical/control-escapes	verify	1263	618	2.04×	-8	-1495
bd:go-vs-rs	canonical/deep	verify	37772	29938	1.26×	-141	-3696
bd:go-vs-rs	canonical/deep-64	verify	16756	11072	1.51×	-65	-1704
bd:go-vs-rs	canonical/escaped-key-order	verify	1121	977	1.15×	+26	-224
bd:go-vs-rs	canonical/large	verify	20033798	23474654	1.17×	+813415	-2018317
bd:go-vs-rs	canonical/long-string	verify	77435	17530	4.42×	-4	-2765
bd:go-vs-rs	canonical/medium	verify	579406	641668	1.11×	+20778	-87207
bd:go-vs-rs	canonical/mixed-prod	verify	177370	137222	1.29×	+2738	-115958
bd:go-vs-rs	canonical/nested-mixed	verify	3118	2250	1.39×	+48	-1334
bd:go-vs-rs	canonical/number-heavy	verify	21592	14567	1.48×	+546	+24718
bd:go-vs-rs	canonical/numeric-boundary	verify	18988	11136	1.71×	+419	+23735
bd:go-vs-rs	canonical/rfc-key-sorting	verify	2844	1337	2.13×	-15	-2575
bd:go-vs-rs	canonical/small	verify	1664	1296	1.28×	+25	-877
bd:go-vs-rs	canonical/surrogate-pair	verify	652	312	2.09×	-4	-713
bd:go-vs-rs	canonical/unicode	verify	1552	1579	1.02×	-6	-1405
bd:go-vs-rs	canonical/verify-whitespace	verify	1597	1371	1.16×	+37	-275

B/op = bytes allocated per operation. allocs/op = heap allocations per operation.
Δ columns show Impl B − Impl A for each pair.

A.2 Full CLI Benchmarks

Table 9: CLI benchmark results (end-to-end, $n = 15$).

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
go:schubfach-vs-bd	array-2048	canonicalize	5.317	7.486	1.41×	schubfach
go:schubfach-vs-bd	array-2048	verify	5.839	7.571	1.30×	schubfach
go:schubfach-vs-bd	array-256	canonicalize	2.285	2.367	1.04×	schubfach
go:schubfach-vs-bd	array-256	verify	2.003	2.259	1.13×	schubfach
go:schubfach-vs-bd	canonical-minimal	canonicalize	1.630	1.452	1.12×	json-canon
go:schubfach-vs-bd	canonical-minimal	verify	1.435	1.587	1.11×	schubfach
go:schubfach-vs-bd	control-escapes	canonicalize	1.639	1.588	1.03×	json-canon
go:schubfach-vs-bd	control-escapes	verify	1.424	1.659	1.17×	schubfach
go:schubfach-vs-bd	deep	canonicalize	1.596	1.769	1.11×	schubfach
go:schubfach-vs-bd	deep	verify	1.653	1.929	1.17×	schubfach
go:schubfach-vs-bd	deep-64	canonicalize	1.498	1.679	1.12×	schubfach
go:schubfach-vs-bd	deep-64	verify	1.669	1.662	1.00×	json-canon
go:schubfach-vs-bd	escaped-key-order	canonicalize	1.552	1.676	1.08×	schubfach
go:schubfach-vs-bd	escaped-key-order	verify	1.686	1.635	1.03×	json-canon
go:schubfach-vs-bd	large	canonicalize	16.427	24.928	1.52×	schubfach
go:schubfach-vs-bd	large	verify	16.316	24.981	1.53×	schubfach
go:schubfach-vs-bd	long-string	canonicalize	1.595	1.787	1.12×	schubfach
go:schubfach-vs-bd	long-string	verify	1.635	1.705	1.04×	schubfach
go:schubfach-vs-bd	medium	canonicalize	2.113	2.657	1.26×	schubfach
go:schubfach-vs-bd	medium	verify	1.999	2.532	1.27×	schubfach
go:schubfach-vs-bd	mixed-prod	canonicalize	1.688	1.879	1.11×	schubfach
go:schubfach-vs-bd	mixed-prod	verify	1.896	1.868	1.01×	json-canon
go:schubfach-vs-bd	nested-mixed	canonicalize	1.519	1.603	1.06×	schubfach
go:schubfach-vs-bd	nested-mixed	verify	1.505	1.707	1.13×	schubfach
go:schubfach-vs-bd	number-heavy	canonicalize	1.476	1.756	1.19×	schubfach
go:schubfach-vs-bd	number-heavy	verify	1.617	1.656	1.02×	schubfach
go:schubfach-vs-bd	numeric-boundary	canonicalize	1.420	1.706	1.20×	schubfach
go:schubfach-vs-bd	numeric-boundary	verify	1.467	1.527	1.04×	schubfach
go:schubfach-vs-bd	rfc-key-sorting	canonicalize	1.482	1.448	1.02×	json-canon
go:schubfach-vs-bd	rfc-key-sorting	verify	1.547	1.603	1.04×	schubfach
go:schubfach-vs-bd	small	canonicalize	1.467	1.551	1.06×	schubfach
go:schubfach-vs-bd	small	verify	1.548	1.826	1.18×	schubfach
go:schubfach-vs-bd	surrogate-pair	canonicalize	1.499	1.605	1.07×	schubfach
go:schubfach-vs-bd	surrogate-pair	verify	1.473	1.709	1.16×	schubfach
go:schubfach-vs-bd	unicode	canonicalize	1.549	1.593	1.03×	schubfach
go:schubfach-vs-bd	unicode	verify	1.430	1.543	1.08×	schubfach
go:schubfach-vs-bd	verify-whitespace	canonicalize	1.549	1.627	1.05×	schubfach
go:schubfach-vs-bd	verify-whitespace	verify	1.498	1.662	1.11×	schubfach
rs:schubfach-vs-bd	array-2048	canonicalize	2.611	4.644	1.78×	schubfach-rs
rs:schubfach-vs-bd	array-2048	verify	2.454	4.454	1.82×	schubfach-rs
rs:schubfach-vs-bd	array-256	canonicalize	0.824	1.192	1.45×	schubfach-rs
rs:schubfach-vs-bd	array-256	verify	0.785	1.169	1.49×	schubfach-rs
rs:schubfach-vs-bd	canonical-minimal	canonicalize	0.561	0.642	1.14×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
rs:schubfach-vs-bd	canonical-minimal	verify	0.586	0.612	1.04×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	canonicalize	0.600	0.615	1.02×	schubfach-rs
rs:schubfach-vs-bd	control-escapes	verify	0.578	0.641	1.11×	schubfach-rs
rs:schubfach-vs-bd	deep	canonicalize	0.621	0.705	1.14×	schubfach-rs
rs:schubfach-vs-bd	deep	verify	0.634	0.696	1.10×	schubfach-rs
rs:schubfach-vs-bd	deep-64	canonicalize	0.596	0.661	1.11×	schubfach-rs
rs:schubfach-vs-bd	deep-64	verify	0.584	0.641	1.10×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	canonicalize	0.547	0.696	1.27×	schubfach-rs
rs:schubfach-vs-bd	escaped-key-order	verify	0.588	0.751	1.28×	schubfach-rs
rs:schubfach-vs-bd	large	canonicalize	7.562	18.613	2.46×	schubfach-rs
rs:schubfach-vs-bd	large	verify	7.447	18.355	2.46×	schubfach-rs
rs:schubfach-vs-bd	long-string	canonicalize	0.631	0.667	1.06×	schubfach-rs
rs:schubfach-vs-bd	long-string	verify	0.620	0.686	1.11×	schubfach-rs
rs:schubfach-vs-bd	medium	canonicalize	0.891	1.402	1.57×	schubfach-rs
rs:schubfach-vs-bd	medium	verify	0.861	1.330	1.55×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	canonicalize	0.679	0.893	1.32×	schubfach-rs
rs:schubfach-vs-bd	mixed-prod	verify	0.678	0.887	1.31×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	canonicalize	0.554	0.669	1.21×	schubfach-rs
rs:schubfach-vs-bd	nested-mixed	verify	0.541	0.719	1.33×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	canonicalize	0.542	0.695	1.28×	schubfach-rs
rs:schubfach-vs-bd	number-heavy	verify	0.578	0.728	1.26×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	canonicalize	0.543	0.741	1.36×	schubfach-rs
rs:schubfach-vs-bd	numeric-boundary	verify	0.589	0.713	1.21×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	canonicalize	0.567	0.628	1.11×	schubfach-rs
rs:schubfach-vs-bd	rfc-key-sorting	verify	0.585	0.646	1.10×	schubfach-rs
rs:schubfach-vs-bd	small	canonicalize	0.591	0.674	1.14×	schubfach-rs
rs:schubfach-vs-bd	small	verify	0.557	0.700	1.26×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	canonicalize	0.559	0.622	1.11×	schubfach-rs
rs:schubfach-vs-bd	surrogate-pair	verify	0.588	0.606	1.03×	schubfach-rs
rs:schubfach-vs-bd	unicode	canonicalize	0.581	0.636	1.09×	schubfach-rs
rs:schubfach-vs-bd	unicode	verify	0.565	0.636	1.12×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	canonicalize	0.549	0.700	1.27×	schubfach-rs
rs:schubfach-vs-bd	verify-whitespace	verify	0.556	0.717	1.29×	schubfach-rs
schubfach:go-vs-rs	array-2048	canonicalize	5.317	2.611	2.04×	schubfach-rs
schubfach:go-vs-rs	array-2048	verify	5.839	2.454	2.38×	schubfach-rs
schubfach:go-vs-rs	array-256	canonicalize	2.285	0.824	2.77×	schubfach-rs
schubfach:go-vs-rs	array-256	verify	2.003	0.785	2.55×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	canonicalize	1.630	0.561	2.90×	schubfach-rs
schubfach:go-vs-rs	canonical-minimal	verify	1.435	0.586	2.45×	schubfach-rs
schubfach:go-vs-rs	control-escapes	canonicalize	1.639	0.600	2.73×	schubfach-rs
schubfach:go-vs-rs	control-escapes	verify	1.424	0.578	2.46×	schubfach-rs
schubfach:go-vs-rs	deep	canonicalize	1.596	0.621	2.57×	schubfach-rs
schubfach:go-vs-rs	deep	verify	1.653	0.634	2.61×	schubfach-rs
schubfach:go-vs-rs	deep-64	canonicalize	1.498	0.596	2.51×	schubfach-rs
schubfach:go-vs-rs	deep-64	verify	1.669	0.584	2.86×	schubfach-rs
schubfach:go-vs-rs	escaped-key-order	canonicalize	1.552	0.547	2.84×	schubfach-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
schubfach:go-vs-rs	escaped-key-order	verify	1.686	0.588	2.86×	schubfach-rs
schubfach:go-vs-rs	large	canonicalize	16.427	7.562	2.17×	schubfach-rs
schubfach:go-vs-rs	large	verify	16.316	7.447	2.19×	schubfach-rs
schubfach:go-vs-rs	long-string	canonicalize	1.595	0.631	2.53×	schubfach-rs
schubfach:go-vs-rs	long-string	verify	1.635	0.620	2.64×	schubfach-rs
schubfach:go-vs-rs	medium	canonicalize	2.113	0.891	2.37×	schubfach-rs
schubfach:go-vs-rs	medium	verify	1.999	0.861	2.32×	schubfach-rs
schubfach:go-vs-rs	mixed-prod	canonicalize	1.688	0.679	2.49×	schubfach-rs
schubfach:go-vs-rs	mixed-prod	verify	1.896	0.678	2.80×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	canonicalize	1.519	0.554	2.74×	schubfach-rs
schubfach:go-vs-rs	nested-mixed	verify	1.505	0.541	2.78×	schubfach-rs
schubfach:go-vs-rs	number-heavy	canonicalize	1.476	0.542	2.72×	schubfach-rs
schubfach:go-vs-rs	number-heavy	verify	1.617	0.578	2.80×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	canonicalize	1.420	0.543	2.62×	schubfach-rs
schubfach:go-vs-rs	numeric-boundary	verify	1.467	0.589	2.49×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	canonicalize	1.482	0.567	2.61×	schubfach-rs
schubfach:go-vs-rs	rfc-key-sorting	verify	1.547	0.585	2.65×	schubfach-rs
schubfach:go-vs-rs	small	canonicalize	1.467	0.591	2.48×	schubfach-rs
schubfach:go-vs-rs	small	verify	1.548	0.557	2.78×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	canonicalize	1.499	0.559	2.68×	schubfach-rs
schubfach:go-vs-rs	surrogate-pair	verify	1.473	0.588	2.50×	schubfach-rs
schubfach:go-vs-rs	unicode	canonicalize	1.549	0.581	2.66×	schubfach-rs
schubfach:go-vs-rs	unicode	verify	1.430	0.565	2.53×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	canonicalize	1.549	0.549	2.82×	schubfach-rs
schubfach:go-vs-rs	verify-whitespace	verify	1.498	0.556	2.69×	schubfach-rs
bd:go-vs-rs	array-2048	canonicalize	7.486	4.644	1.61×	json-canon-rs
bd:go-vs-rs	array-2048	verify	7.571	4.454	1.70×	json-canon-rs
bd:go-vs-rs	array-256	canonicalize	2.367	1.192	1.99×	json-canon-rs
bd:go-vs-rs	array-256	verify	2.259	1.169	1.93×	json-canon-rs
bd:go-vs-rs	canonical-minimal	canonicalize	1.452	0.642	2.26×	json-canon-rs
bd:go-vs-rs	canonical-minimal	verify	1.587	0.612	2.59×	json-canon-rs
bd:go-vs-rs	control-escapes	canonicalize	1.588	0.615	2.58×	json-canon-rs
bd:go-vs-rs	control-escapes	verify	1.659	0.641	2.59×	json-canon-rs
bd:go-vs-rs	deep	canonicalize	1.769	0.705	2.51×	json-canon-rs
bd:go-vs-rs	deep	verify	1.929	0.696	2.77×	json-canon-rs
bd:go-vs-rs	deep-64	canonicalize	1.679	0.661	2.54×	json-canon-rs
bd:go-vs-rs	deep-64	verify	1.662	0.641	2.59×	json-canon-rs
bd:go-vs-rs	escaped-key-order	canonicalize	1.676	0.696	2.41×	json-canon-rs
bd:go-vs-rs	escaped-key-order	verify	1.635	0.751	2.18×	json-canon-rs
bd:go-vs-rs	large	canonicalize	24.928	18.613	1.34×	json-canon-rs
bd:go-vs-rs	large	verify	24.981	18.355	1.36×	json-canon-rs
bd:go-vs-rs	long-string	canonicalize	1.787	0.667	2.68×	json-canon-rs
bd:go-vs-rs	long-string	verify	1.705	0.686	2.48×	json-canon-rs
bd:go-vs-rs	medium	canonicalize	2.657	1.402	1.89×	json-canon-rs
bd:go-vs-rs	medium	verify	2.532	1.330	1.90×	json-canon-rs
bd:go-vs-rs	mixed-prod	canonicalize	1.879	0.893	2.10×	json-canon-rs

Continued on next page

Pair	Workload	Mode	Impl A (ms)	Impl B (ms)	Speedup	Winner
bd:go-vs-rs	mixed-prod	verify	1.868	0.887	2.11×	json-canon-rs
bd:go-vs-rs	nested-mixed	canonicalize	1.603	0.669	2.40×	json-canon-rs
bd:go-vs-rs	nested-mixed	verify	1.707	0.719	2.37×	json-canon-rs
bd:go-vs-rs	number-heavy	canonicalize	1.756	0.695	2.53×	json-canon-rs
bd:go-vs-rs	number-heavy	verify	1.656	0.728	2.27×	json-canon-rs
bd:go-vs-rs	numeric-boundary	canonicalize	1.706	0.741	2.30×	json-canon-rs
bd:go-vs-rs	numeric-boundary	verify	1.527	0.713	2.14×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	canonicalize	1.448	0.628	2.31×	json-canon-rs
bd:go-vs-rs	rfc-key-sorting	verify	1.603	0.646	2.48×	json-canon-rs
bd:go-vs-rs	small	canonicalize	1.551	0.674	2.30×	json-canon-rs
bd:go-vs-rs	small	verify	1.826	0.700	2.61×	json-canon-rs
bd:go-vs-rs	surrogate-pair	canonicalize	1.605	0.622	2.58×	json-canon-rs
bd:go-vs-rs	surrogate-pair	verify	1.709	0.606	2.82×	json-canon-rs
bd:go-vs-rs	unicode	canonicalize	1.593	0.636	2.50×	json-canon-rs
bd:go-vs-rs	unicode	verify	1.543	0.636	2.43×	json-canon-rs
bd:go-vs-rs	verify-whitespace	canonicalize	1.627	0.700	2.32×	json-canon-rs
bd:go-vs-rs	verify-whitespace	verify	1.662	0.717	2.32×	json-canon-rs

Speedup = slower/faster mean wall time. $n = 15$ repetitions, 3 warmup runs discarded.

A.3 Full Statistical Comparisons

Table 10: Statistical comparisons with Benjamini-Hochberg corrected p-values.

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
go:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach*	1.41×	[1.310, 1.527]	0.0002	0.0003	-2.98	0.5235	10.1
rs:schubfach-vs-bd	e2e	canonicalize	array-2048	schubfach-rs*	1.78×	[1.589, 2.010]	0.0002	0.0003	-3.30	0.4823	18.9
schubfach:go-vs-rs	e2e	canonicalize	array-2048	schubfach-rs*	2.04×	[1.844, 2.261]	0.0002	0.0003	5.38	0.4823	18.9
bd:go-vs-rs	e2e	canonicalize	array-2048	json-canon-rs*	1.61×	[1.463, 1.769]	0.0002	0.0003	3.51	0.7259	16.0
go:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach	1.04×	[0.948, 1.150]	0.4909	0.5205	-0.25	0.3029	13.6
rs:schubfach-vs-bd	e2e	canonicalize	array-256	schubfach-rs*	1.45×	[1.306, 1.593]	0.0002	0.0003	-2.55	0.1061	13.2
schubfach:go-vs-rs	e2e	canonicalize	array-256	schubfach-rs*	2.77×	[2.513, 3.016]	0.0002	0.0003	6.44	0.1061	13.2
bd:go-vs-rs	e2e	canonicalize	array-256	json-canon-rs*	1.99×	[1.802, 2.210]	0.0002	0.0003	4.29	0.1745	15.0
go:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	json-canon	1.12×	[0.986, 1.266]	0.0900	0.1068	0.65	0.2976	21.0
rs:schubfach-vs-bd	e2e	canonicalize	canonical-minimal	schubfach-rs*	1.14×	[1.087, 1.216]	0.0002	0.0003	-1.72	0.0523	9.5
schubfach:go-vs-rs	e2e	canonicalize	canonical-minimal	schubfach-rs*	2.90×	[2.636, 3.148]	0.0002	0.0003	5.97	0.0523	9.5
bd:go-vs-rs	e2e	canonicalize	canonical-minimal	json-canon-rs*	2.26×	[2.046, 2.512]	0.0002	0.0003	3.82	0.0400	6.4
go:schubfach-vs-bd	e2e	canonicalize	control-escapes	json-canon	1.03×	[0.912, 1.174]	0.6453	0.6663	0.17	0.3278	21.1
rs:schubfach-vs-bd	e2e	canonicalize	control-escapes	schubfach-rs	1.02×	[0.941, 1.091]	0.5377	0.5663	-0.23	0.0562	9.6
schubfach:go-vs-rs	e2e	canonicalize	control-escapes	schubfach-rs*	2.73×	[2.479, 2.968]	0.0002	0.0003	5.66	0.0562	9.6
bd:go-vs-rs	e2e	canonicalize	control-escapes	json-canon-rs*	2.58×	[2.290, 2.883]	0.0002	0.0003	4.11	0.0708	11.8
go:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach	1.12×	[0.988, 1.267]	0.1078	0.1261	-0.62	0.2649	18.1
rs:schubfach-vs-bd	e2e	canonicalize	deep-64	schubfach-rs*	1.11×	[1.030, 1.215]	0.0156	0.0194	-0.86	0.0401	6.9
schubfach:go-vs-rs	e2e	canonicalize	deep-64	schubfach-rs*	2.51×	[2.286, 2.759]	0.0002	0.0003	4.76	0.0401	6.9
bd:go-vs-rs	e2e	canonicalize	deep-64	json-canon-rs*	2.54×	[2.229, 2.825]	0.0002	0.0003	4.37	0.1004	15.5
go:schubfach-vs-bd	e2e	canonicalize	deep	schubfach	1.11×	[0.960, 1.276]	0.1778	0.2013	-0.50	0.3331	21.3
rs:schubfach-vs-bd	e2e	canonicalize	deep	schubfach-rs	1.14×	[1.019, 1.268]	0.0424	0.0511	-0.79	0.0775	12.8
schubfach:go-vs-rs	e2e	canonicalize	deep	schubfach-rs*	2.57×	[2.270, 2.877]	0.0002	0.0003	4.03	0.0775	12.8
bd:go-vs-rs	e2e	canonicalize	deep	json-canon-rs*	2.51×	[2.199, 2.876]	0.0002	0.0003	3.99	0.1283	18.6

Continued on next page

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
go:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach	1.08×	[0.960, 1.235]	0.2513	0.2825	-0.43	0.2787	18.4
rs:schubfach-vs-bd	e2e	canonicalize	escaped-key-order	schubfach-rs*	1.27×	[1.214, 1.348]	0.0002	0.0003	-3.44	0.0531	9.9
schubfach:go-vs-rs	e2e	canonicalize	escaped-key-order	schubfach-rs*	2.84×	[2.542, 3.108]	0.0002	0.0003	5.01	0.0531	9.9
bd:go-vs-rs	e2e	canonicalize	escaped-key-order	json-canon-rs*	2.41×	[2.197, 2.632]	0.0002	0.0003	4.55	0.0311	4.6
go:schubfach-vs-bd	e2e	canonicalize	large	schubfach*	1.52×	[1.485, 1.549]	0.0002	0.0003	-12.68	0.4621	2.9
rs:schubfach-vs-bd	e2e	canonicalize	large	schubfach-rs*	2.46×	[2.360, 2.540]	0.0002	0.0003	-20.08	0.5016	6.8
schubfach:go-vs-rs	e2e	canonicalize	large	schubfach-rs*	2.17×	[2.086, 2.239]	0.0002	0.0003	18.38	0.5016	6.8
bd:go-vs-rs	e2e	canonicalize	large	json-canon-rs*	1.34×	[1.308, 1.368]	0.0002	0.0003	8.76	0.5951	3.3
go:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach	1.12×	[0.983, 1.261]	0.1032	0.1216	-0.62	0.2672	17.1
rs:schubfach-vs-bd	e2e	canonicalize	long-string	schubfach-rs	1.06×	[0.981, 1.123]	0.1128	0.1305	-0.59	0.0578	9.4
schubfach:go-vs-rs	e2e	canonicalize	long-string	schubfach-rs*	2.53×	[2.284, 2.762]	0.0002	0.0003	4.99	0.0578	9.4
bd:go-vs-rs	e2e	canonicalize	long-string	json-canon-rs*	2.68×	[2.391, 2.953]	0.0002	0.0003	4.48	0.0652	10.0
go:schubfach-vs-bd	e2e	canonicalize	medium	schubfach*	1.26×	[1.104, 1.439]	0.0030	0.0039	-1.20	0.4592	22.2
rs:schubfach-vs-bd	e2e	canonicalize	medium	schubfach-rs*	1.57×	[1.451, 1.731]	0.0002	0.0003	-3.61	0.1225	14.1
schubfach:go-vs-rs	e2e	canonicalize	medium	schubfach-rs*	2.37×	[2.092, 2.691]	0.0002	0.0003	3.64	0.1225	14.1
bd:go-vs-rs	e2e	canonicalize	medium	json-canon-rs*	1.89×	[1.709, 2.090]	0.0002	0.0003	3.74	0.1581	11.5
go:schubfach-vs-bd	e2e	canonicalize	mixed-prod	schubfach	1.11×	[0.974, 1.291]	0.1562	0.1781	-0.52	0.3365	20.4
rs:schubfach-vs-bd	e2e	canonicalize	mixed-prod	schubfach-rs*	1.32×	[1.215, 1.440]	0.0002	0.0003	-2.27	0.0880	13.2
schubfach:go-vs-rs	e2e	canonicalize	mixed-prod	schubfach-rs*	2.49×	[2.218, 2.808]	0.0002	0.0003	4.11	0.0880	13.2
bd:go-vs-rs	e2e	canonicalize	mixed-prod	json-canon-rs*	2.10×	[1.887, 2.376]	0.0002	0.0003	3.47	0.1007	11.5
go:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach	1.06×	[0.917, 1.223]	0.4903	0.5205	-0.26	0.2870	19.3
rs:schubfach-vs-bd	e2e	canonicalize	nested-mixed	schubfach-rs*	1.21×	[1.102, 1.320]	0.0014	0.0018	-1.36	0.0507	9.4
schubfach:go-vs-rs	e2e	canonicalize	nested-mixed	schubfach-rs*	2.74×	[2.455, 3.017]	0.0002	0.0003	4.69	0.0507	9.4
bd:go-vs-rs	e2e	canonicalize	nested-mixed	json-canon-rs*	2.40×	[2.090, 2.741]	0.0002	0.0003	3.55	0.1082	16.5
go:schubfach-vs-bd	e2e	canonicalize	number-heavy	schubfach*	1.19×	[1.048, 1.341]	0.0138	0.0174	-1.00	0.2711	18.8
rs:schubfach-vs-bd	e2e	canonicalize	number-heavy	schubfach-rs*	1.28×	[1.174, 1.412]	0.0002	0.0003	-1.92	0.0702	13.2
schubfach:go-vs-rs	e2e	canonicalize	number-heavy	schubfach-rs*	2.72×	[2.440, 3.042]	0.0002	0.0003	4.72	0.0702	13.2
bd:go-vs-rs	e2e	canonicalize	number-heavy	json-canon-rs*	2.53×	[2.260, 2.777]	0.0002	0.0003	4.96	0.0882	13.0
go:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach*	1.20×	[1.044, 1.353]	0.0140	0.0176	-0.97	0.2548	18.3
rs:schubfach-vs-bd	e2e	canonicalize	numeric-boundary	schubfach-rs*	1.36×	[1.272, 1.461]	0.0002	0.0003	-3.11	0.0583	11.0
schubfach:go-vs-rs	e2e	canonicalize	numeric-boundary	schubfach-rs*	2.62×	[2.366, 2.899]	0.0002	0.0003	4.75	0.0583	11.0
bd:go-vs-rs	e2e	canonicalize	numeric-boundary	json-canon-rs*	2.30×	[2.057, 2.532]	0.0002	0.0003	4.03	0.0684	9.4
go:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	json-canon	1.02×	[0.886, 1.181]	0.7630	0.7778	0.11	0.2843	20.1
rs:schubfach-vs-bd	e2e	canonicalize	rfc-key-sorting	schubfach-rs*	1.11×	[1.006, 1.207]	0.0400	0.0486	-0.78	0.0537	9.7
schubfach:go-vs-rs	e2e	canonicalize	rfc-key-sorting	schubfach-rs*	2.61×	[2.314, 2.945]	0.0002	0.0003	3.77	0.0537	9.7
bd:go-vs-rs	e2e	canonicalize	rfc-key-sorting	json-canon-rs*	2.31×	[2.060, 2.620]	0.0002	0.0003	3.87	0.0949	15.5
go:schubfach-vs-bd	e2e	canonicalize	small	schubfach	1.06×	[0.922, 1.214]	0.4501	0.4886	-0.28	0.3144	21.9
rs:schubfach-vs-bd	e2e	canonicalize	small	schubfach-rs*	1.14×	[1.062, 1.211]	0.0018	0.0023	-1.35	0.0424	7.3
schubfach:go-vs-rs	e2e	canonicalize	small	schubfach-rs*	2.48×	[2.217, 2.770]	0.0002	0.0003	3.91	0.0424	7.3
bd:go-vs-rs	e2e	canonicalize	small	json-canon-rs*	2.30×	[2.061, 2.558]	0.0002	0.0003	4.12	0.0758	11.5
go:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach	1.07×	[0.948, 1.212]	0.2969	0.3303	-0.39	0.2473	16.9
rs:schubfach-vs-bd	e2e	canonicalize	surrogate-pair	schubfach-rs*	1.11×	[1.038, 1.189]	0.0066	0.0084	-1.09	0.0548	10.0
schubfach:go-vs-rs	e2e	canonicalize	surrogate-pair	schubfach-rs*	2.68×	[2.438, 2.937]	0.0002	0.0003	5.25	0.0548	10.0
bd:go-vs-rs	e2e	canonicalize	surrogate-pair	json-canon-rs*	2.58×	[2.310, 2.834]	0.0002	0.0003	4.64	0.0611	10.0
go:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach	1.03×	[0.898, 1.157]	0.6703	0.6899	-0.16	0.2291	15.1
rs:schubfach-vs-bd	e2e	canonicalize	unicode	schubfach-rs*	1.09×	[1.024, 1.170]	0.0192	0.0238	-0.93	0.0508	8.9
schubfach:go-vs-rs	e2e	canonicalize	unicode	schubfach-rs*	2.66×	[2.433, 2.873]	0.0002	0.0003	5.83	0.0508	8.9
bd:go-vs-rs	e2e	canonicalize	unicode	json-canon-rs*	2.50×	[2.207, 2.776]	0.0002	0.0003	4.01	0.0659	10.6
go:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach	1.05×	[0.933, 1.183]	0.4443	0.4840	-0.28	0.2620	17.3
rs:schubfach-vs-bd	e2e	canonicalize	verify-whitespace	schubfach-rs*	1.27×	[1.202, 1.365]	0.0002	0.0003	-2.74	0.0600	11.2
schubfach:go-vs-rs	e2e	canonicalize	verify-whitespace	schubfach-rs*	2.82×	[2.524, 3.081]	0.0002	0.0003	5.26	0.0600	11.2
bd:go-vs-rs	e2e	canonicalize	verify-whitespace	json-canon-rs*	2.32×	[2.106, 2.542]	0.0002	0.0003	4.49	0.0496	7.2
go:schubfach-vs-bd	e2e	verify	array-2048	schubfach*	1.30×	[1.215, 1.389]	0.0002	0.0003	-2.59	0.4550	8.0
rs:schubfach-vs-bd	e2e	verify	array-2048	schubfach-rs*	1.82×	[1.637, 2.036]	0.0002	0.0003	-3.70	0.4360	18.2
schubfach:go-vs-rs	e2e	verify	array-2048	schubfach-rs*	2.38×	[2.165, 2.622]	0.0002	0.0003	7.60	0.4360	18.2
bd:go-vs-rs	e2e	verify	array-2048	json-canon-rs*	1.70×	[1.541, 1.836]	0.0002	0.0003	4.23	0.6270	14.4
go:schubfach-vs-bd	e2e	verify	array-256	schubfach	1.13×	[0.981, 1.309]	0.1224	0.1411	-0.58	0.4099	20.9
rs:schubfach-vs-bd	e2e	verify	array-256	schubfach-rs*	1.49×	[1.352, 1.627]	0.0002	0.0003	-2.95	0.1125	14.7
schubfach:go-vs-rs	e2e	verify	array-256	schubfach-rs*	2.55×	[2.270, 2.884]	0.0002	0.0003	4.05	0.1125	14.7
bd:go-vs-rs	e2e	verify	array-256	json-canon-rs*	1.93×	[1.724, 2.190]	0.0002	0.0003	3.12	0.1454	12.7
go:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach	1.11×	[0.981, 1.239]	0.1092	0.1272	-0.60	0.2397	17.1
rs:schubfach-vs-bd	e2e	verify	canonical-minimal	schubfach-rs	1.04×	[0.988, 1.089]	0.1126	0.1305	-0.61	0.0395	6.9
schubfach:go-vs-rs	e2e	verify	canonical-minimal	schubfach-rs*	2.45×	[2.236, 2.668]	0.0002	0.0003	4.94	0.0395	6.9

Continued on next page

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
bd:go-vs-rs	e2e	verify	canonical-minimal	json-canon-rs*	2.59×	[2.374, 2.845]	0.0002	0.0003	5.11	0.0434	7.3
go:schubfach-vs-bd	e2e	verify	control-escapes	schubfach*	1.17×	[1.035, 1.295]	0.0152	0.0190	-0.95	0.2531	18.2
rs:schubfach-vs-bd	e2e	verify	control-escapes	schubfach-rs	1.11×	[1.012, 1.253]	0.0688	0.0823	-0.70	0.0893	15.8
schubfach:go-vs-rs	e2e	verify	control-escapes	schubfach-rs*	2.46×	[2.201, 2.766]	0.0002	0.0003	4.46	0.0893	15.8
bd:go-vs-rs	e2e	verify	control-escapes	json-canon-rs*	2.59×	[2.307, 2.828]	0.0002	0.0003	5.59	0.0910	14.5
go:schubfach-vs-bd	e2e	verify	deep-64	json-canon	1.00×	[0.897, 1.113]	0.9392	0.9452	0.03	0.2644	16.3
rs:schubfach-vs-bd	e2e	verify	deep-64	schubfach-rs*	1.10×	[1.042, 1.174]	0.0026	0.0034	-1.11	0.0594	10.4
schubfach:go-vs-rs	e2e	verify	deep-64	schubfach-rs*	2.86×	[2.571, 3.100]	0.0002	0.0003	5.88	0.0594	10.4
bd:go-vs-rs	e2e	verify	deep-64	json-canon-rs*	2.59×	[2.345, 2.791]	0.0002	0.0003	5.39	0.0424	6.8
go:schubfach-vs-bd	e2e	verify	deep	schubfach*	1.17×	[1.045, 1.291]	0.0126	0.0160	-0.98	0.2399	14.8
rs:schubfach-vs-bd	e2e	verify	deep	schubfach-rs	1.10×	[1.010, 1.193]	0.0486	0.0583	-0.74	0.0399	6.4
schubfach:go-vs-rs	e2e	verify	deep	schubfach-rs*	2.61×	[2.405, 2.805]	0.0002	0.0003	5.93	0.0399	6.4
bd:go-vs-rs	e2e	verify	deep	json-canon-rs*	2.77×	[2.462, 3.082]	0.0002	0.0003	5.19	0.1102	16.2
go:schubfach-vs-bd	e2e	verify	escaped-key-order	json-canon	1.03×	[0.935, 1.164]	0.5807	0.6075	0.20	0.3079	19.3
rs:schubfach-vs-bd	e2e	verify	escaped-key-order	schubfach-rs*	1.28×	[1.111, 1.806]	0.0018	0.0023	-0.72	0.0652	11.3
schubfach:go-vs-rs	e2e	verify	escaped-key-order	schubfach-rs*	2.86×	[2.616, 3.061]	0.0002	0.0003	7.78	0.0652	11.3
bd:go-vs-rs	e2e	verify	escaped-key-order	json-canon-rs*	2.18×	[1.574, 2.550]	0.0002	0.0003	2.84	0.3140	42.7
go:schubfach-vs-bd	e2e	verify	large	schubfach*	1.53×	[1.488, 1.576]	0.0002	0.0003	-11.04	0.8674	5.4
rs:schubfach-vs-bd	e2e	verify	large	schubfach-rs*	2.46×	[2.241, 2.608]	0.0002	0.0003	-13.67	1.0924	15.0
schubfach:go-vs-rs	e2e	verify	large	schubfach-rs*	2.19×	[2.003, 2.337]	0.0002	0.0003	8.99	1.0924	15.0
bd:go-vs-rs	e2e	verify	large	json-canon-rs*	1.36×	[1.340, 1.383]	0.0002	0.0003	12.52	0.2826	1.6
go:schubfach-vs-bd	e2e	verify	long-string	schubfach	1.04×	[0.935, 1.168]	0.4789	0.5112	-0.26	0.2789	17.4
rs:schubfach-vs-bd	e2e	verify	long-string	schubfach-rs*	1.11×	[1.044, 1.186]	0.0054	0.0070	-1.10	0.0602	9.9
schubfach:go-vs-rs	e2e	verify	long-string	schubfach-rs*	2.64×	[2.399, 2.895]	0.0002	0.0003	5.03	0.0602	9.9
bd:go-vs-rs	e2e	verify	long-string	json-canon-rs*	2.48×	[2.273, 2.707]	0.0002	0.0003	5.38	0.0599	8.9
go:schubfach-vs-bd	e2e	verify	medium	schubfach*	1.27×	[1.134, 1.407]	0.0012	0.0016	-1.50	0.3463	17.7
rs:schubfach-vs-bd	e2e	verify	medium	schubfach-rs*	1.55×	[1.392, 1.727]	0.0002	0.0003	-2.48	0.1015	12.1
schubfach:go-vs-rs	e2e	verify	medium	schubfach-rs*	2.32×	[2.106, 2.585]	0.0002	0.0003	4.46	0.1015	12.1
bd:go-vs-rs	e2e	verify	medium	json-canon-rs*	1.90×	[1.689, 2.135]	0.0002	0.0003	3.85	0.2478	19.0
go:schubfach-vs-bd	e2e	verify	mixed-prod	json-canon	1.01×	[0.895, 1.152]	0.8280	0.8386	0.08	0.2817	15.4
rs:schubfach-vs-bd	e2e	verify	mixed-prod	schubfach-rs*	1.31×	[1.193, 1.423]	0.0002	0.0003	-2.06	0.0785	11.8
schubfach:go-vs-rs	e2e	verify	mixed-prod	schubfach-rs*	2.80×	[2.508, 3.154]	0.0002	0.0003	4.29	0.0785	11.8
bd:go-vs-rs	e2e	verify	mixed-prod	json-canon-rs*	2.11×	[1.917, 2.342]	0.0002	0.0003	4.53	0.1202	13.9
go:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach	1.13×	[0.992, 1.305]	0.0878	0.1046	-0.64	0.3073	20.9
rs:schubfach-vs-bd	e2e	verify	nested-mixed	schubfach-rs*	1.33×	[1.229, 1.458]	0.0002	0.0003	-2.47	0.0810	15.3
schubfach:go-vs-rs	e2e	verify	nested-mixed	schubfach-rs*	2.78×	[2.471, 3.162]	0.0002	0.0003	4.29	0.0810	15.3
bd:go-vs-rs	e2e	verify	nested-mixed	json-canon-rs*	2.37×	[2.152, 2.634]	0.0002	0.0003	4.29	0.0625	8.9
go:schubfach-vs-bd	e2e	verify	number-heavy	schubfach	1.02×	[0.905, 1.156]	0.7075	0.7258	-0.14	0.2023	12.8
rs:schubfach-vs-bd	e2e	verify	number-heavy	schubfach-rs*	1.26×	[1.174, 1.332]	0.0002	0.0003	-2.44	0.0446	7.9
schubfach:go-vs-rs	e2e	verify	number-heavy	schubfach-rs*	2.80×	[2.596, 2.986]	0.0002	0.0003	7.09	0.0446	7.9
bd:go-vs-rs	e2e	verify	number-heavy	json-canon-rs*	2.27×	[2.020, 2.549]	0.0002	0.0003	3.65	0.0749	10.5
go:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach	1.04×	[0.899, 1.203]	0.5929	0.6182	-0.19	0.2984	20.8
rs:schubfach-vs-bd	e2e	verify	numeric-boundary	schubfach-rs*	1.21×	[1.104, 1.307]	0.0004	0.0005	-1.55	0.0692	12.0
schubfach:go-vs-rs	e2e	verify	numeric-boundary	schubfach-rs*	2.49×	[2.201, 2.775]	0.0002	0.0003	4.05	0.0692	12.0
bd:go-vs-rs	e2e	verify	numeric-boundary	json-canon-rs*	2.14×	[1.907, 2.420]	0.0002	0.0003	3.44	0.0885	12.7
go:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach	1.04×	[0.925, 1.182]	0.5729	0.6013	-0.20	0.2628	17.4
rs:schubfach-vs-bd	e2e	verify	rfc-key-sorting	schubfach-rs*	1.10×	[1.051, 1.178]	0.0006	0.0008	-1.24	0.0478	8.4
schubfach:go-vs-rs	e2e	verify	rfc-key-sorting	schubfach-rs*	2.65×	[2.381, 2.869]	0.0002	0.0003	5.10	0.0478	8.4
bd:go-vs-rs	e2e	verify	rfc-key-sorting	json-canon-rs*	2.48×	[2.235, 2.717]	0.0002	0.0003	4.58	0.0509	8.1
go:schubfach-vs-bd	e2e	verify	small	schubfach*	1.18×	[1.062, 1.315]	0.0064	0.0082	-1.07	0.2786	18.4
rs:schubfach-vs-bd	e2e	verify	small	schubfach-rs*	1.26×	[1.166, 1.361]	0.0002	0.0003	-2.08	0.0684	12.6
schubfach:go-vs-rs	e2e	verify	small	schubfach-rs*	2.78×	[2.485, 3.070]	0.0002	0.0003	4.89	0.0684	12.6
bd:go-vs-rs	e2e	verify	small	json-canon-rs*	2.61×	[2.394, 2.814]	0.0002	0.0003	6.45	0.0691	10.1
go:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach*	1.16×	[1.025, 1.310]	0.0296	0.0362	-0.86	0.3351	23.3
rs:schubfach-vs-bd	e2e	verify	surrogate-pair	schubfach-rs	1.03×	[0.956, 1.076]	0.3305	0.3638	-0.36	0.0398	6.9
schubfach:go-vs-rs	e2e	verify	surrogate-pair	schubfach-rs*	2.50×	[2.205, 2.792]	0.0002	0.0003	3.71	0.0398	6.9
bd:go-vs-rs	e2e	verify	surrogate-pair	json-canon-rs*	2.82×	[2.597, 3.016]	0.0002	0.0003	7.60	0.0563	9.5
go:schubfach-vs-bd	e2e	verify	unicode	schubfach	1.08×	[0.944, 1.221]	0.2859	0.3203	-0.39	0.2651	19.0
rs:schubfach-vs-bd	e2e	verify	unicode	schubfach-rs*	1.12×	[1.032, 1.225]	0.0122	0.0155	-0.95	0.0620	11.2
schubfach:go-vs-rs	e2e	verify	unicode	schubfach-rs*	2.53×	[2.261, 2.810]	0.0002	0.0003	4.49	0.0620	11.2
bd:go-vs-rs	e2e	verify	unicode	json-canon-rs*	2.43×	[2.137, 2.705]	0.0002	0.0003	4.03	0.0834	13.4
go:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach	1.11×	[0.967, 1.255]	0.1408	0.1611	-0.55	0.2836	19.4
rs:schubfach-vs-bd	e2e	verify	verify-whitespace	schubfach-rs*	1.29×	[1.215, 1.385]	0.0002	0.0003	-2.68	0.0518	9.5

Continued on next page

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
schubfach:go-vs-rs	e2e	verify	verify-whitespace	schubfach-rs*	2.69×	[2.422, 2.975]	0.0002	0.0003	4.62	0.0518	9.5
bd:go-vs-rs	e2e	verify	verify-whitespace	json-canon-rs*	2.32×	[2.067, 2.539]	0.0002	0.0003	4.21	0.0671	9.6
go:schubfach-vs-bd	api	canonicalize	array-2048	schubfach*	1.60×	[1.595, 1.608]	0.0002	0.0003	-92.35	0.0132	0.7
rs:schubfach-vs-bd	api	canonicalize	array-2048	schubfach-rs*	2.90×	[2.855, 2.931]	0.0002	0.0003	-101.90	0.0287	2.6
schubfach:go-vs-rs	api	canonicalize	array-2048	schubfach-rs*	1.86×	[1.826, 1.876]	0.0002	0.0003	52.59	0.0287	2.6
bd:go-vs-rs	api	canonicalize	array-2048	json-canon-rs*	1.03×	[1.020, 1.029]	0.0002	0.0003	4.83	0.0220	0.7
go:schubfach-vs-bd	api	canonicalize	array-256	schubfach*	1.54×	[1.531, 1.550]	0.0002	0.0003	-61.98	0.0030	1.2
rs:schubfach-vs-bd	api	canonicalize	array-256	schubfach-rs*	2.55×	[2.478, 2.600]	0.0002	0.0003	-50.86	0.0069	5.0
schubfach:go-vs-rs	api	canonicalize	array-256	schubfach-rs*	1.77×	[1.717, 1.806]	0.0002	0.0003	24.93	0.0069	5.0
bd:go-vs-rs	api	canonicalize	array-256	json-canon-rs*	1.07×	[1.064, 1.074]	0.0002	0.0003	12.06	0.0027	0.8
go:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach*	1.52×	[1.508, 1.539]	0.0002	0.0003	-30.19	0.0000	0.7
rs:schubfach-vs-bd	api	canonicalize	canonical-minimal	schubfach-rs*	1.61×	[1.466, 1.898]	0.0002	0.0003	-3.80	0.0001	22.9
schubfach:go-vs-rs	api	canonicalize	canonical-minimal	schubfach-rs	1.01×	[0.923, 1.154]	0.9154	0.9242	0.04	0.0001	22.9
bd:go-vs-rs	api	canonicalize	canonical-minimal	json-canon*	1.06×	[1.019, 1.140]	0.0200	0.0247	-0.88	0.0000	2.0
go:schubfach-vs-bd	api	canonicalize	control-escapes	json-canon*	1.07×	[1.065, 1.079]	0.0002	0.0003	8.65	0.0000	0.8
rs:schubfach-vs-bd	api	canonicalize	control-escapes	json-canon-rs*	1.81×	[1.514, 1.914]	0.0004	0.0005	4.27	0.0000	5.0
schubfach:go-vs-rs	api	canonicalize	control-escapes	schubfach-rs*	1.16×	[1.102, 1.361]	0.0002	0.0003	1.53	0.0002	18.3
bd:go-vs-rs	api	canonicalize	control-escapes	json-canon-rs*	1.96×	[1.894, 1.992]	0.0002	0.0003	32.00	0.0000	5.0
go:schubfach-vs-bd	api	canonicalize	deep-64	json-canon	1.00×	[0.997, 1.007]	0.4695	0.5062	0.33	0.0001	0.8
rs:schubfach-vs-bd	api	canonicalize	deep-64	json-canon-rs*	1.16×	[1.097, 1.191]	0.0002	0.0003	3.34	0.0001	1.4
schubfach:go-vs-rs	api	canonicalize	deep-64	schubfach-rs*	1.25×	[1.225, 1.331]	0.0002	0.0003	6.07	0.0008	7.3
bd:go-vs-rs	api	canonicalize	deep-64	json-canon-rs*	1.46×	[1.443, 1.466]	0.0002	0.0003	44.49	0.0001	1.4
go:schubfach-vs-bd	api	canonicalize	deep	schubfach	1.00×	[0.995, 1.007]	0.7778	0.7903	-0.13	0.0003	0.9
rs:schubfach-vs-bd	api	canonicalize	deep	json-canon-rs*	1.04×	[1.016, 1.083]	0.0146	0.0183	1.13	0.0008	3.2
schubfach:go-vs-rs	api	canonicalize	deep	schubfach-rs*	1.14×	[1.099, 1.168]	0.0002	0.0003	4.13	0.0015	6.0
bd:go-vs-rs	api	canonicalize	deep	json-canon-rs*	1.19×	[1.162, 1.206]	0.0002	0.0003	10.14	0.0008	3.2
go:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach*	1.60×	[1.591, 1.609]	0.0002	0.0003	-64.73	0.0000	0.9
rs:schubfach-vs-bd	api	canonicalize	escaped-key-order	schubfach-rs*	1.55×	[1.444, 1.792]	0.0002	0.0003	-4.54	0.0001	19.8
schubfach:go-vs-rs	api	canonicalize	escaped-key-order	schubfach-rs*	1.14×	[1.063, 1.319]	0.0002	0.0003	1.22	0.0001	19.8
bd:go-vs-rs	api	canonicalize	escaped-key-order	json-canon-rs*	1.17×	[1.128, 1.194]	0.0002	0.0003	5.73	0.0000	5.3
go:schubfach-vs-bd	api	canonicalize	large	schubfach*	1.72×	[1.714, 1.728]	0.0002	0.0003	-101.13	0.0602	0.6
rs:schubfach-vs-bd	api	canonicalize	large	schubfach-rs*	4.15×	[4.119, 4.177]	0.0002	0.0003	-137.80	0.0506	1.1
schubfach:go-vs-rs	api	canonicalize	large	schubfach-rs*	2.11×	[2.102, 2.127]	0.0002	0.0003	113.23	0.0506	1.1
bd:go-vs-rs	api	canonicalize	large	json-canon*	1.14×	[1.134, 1.146]	0.0002	0.0003	-20.00	0.1043	0.6
go:schubfach-vs-bd	api	canonicalize	long-string	json-canon	1.01×	[0.997, 1.023]	0.1760	0.1999	0.63	0.0013	2.1
rs:schubfach-vs-bd	api	canonicalize	long-string	json-canon-rs*	1.11×	[1.071, 1.122]	0.0002	0.0003	4.11	0.0003	2.1
schubfach:go-vs-rs	api	canonicalize	long-string	schubfach-rs*	4.07×	[4.016, 4.197]	0.0002	0.0003	66.22	0.0006	3.7
bd:go-vs-rs	api	canonicalize	long-string	json-canon-rs*	4.46×	[4.404, 4.526]	0.0002	0.0003	63.75	0.0003	2.1
go:schubfach-vs-bd	api	canonicalize	medium	schubfach*	1.70×	[1.690, 1.712]	0.0002	0.0003	-62.02	0.0028	1.0
rs:schubfach-vs-bd	api	canonicalize	medium	schubfach-rs*	3.40×	[3.129, 3.505]	0.0002	0.0003	-39.04	0.0156	10.5
schubfach:go-vs-rs	api	canonicalize	medium	schubfach-rs*	1.84×	[1.671, 1.894]	0.0002	0.0003	13.89	0.0156	10.5
bd:go-vs-rs	api	canonicalize	medium	json-canon*	1.09×	[1.083, 1.096]	0.0002	0.0003	-11.56	0.0047	1.0
go:schubfach-vs-bd	api	canonicalize	mixed-prod	schubfach*	1.32×	[1.313, 1.333]	0.0002	0.0003	-29.25	0.0006	0.6
rs:schubfach-vs-bd	api	canonicalize	mixed-prod	schubfach-rs*	2.04×	[2.014, 2.063]	0.0002	0.0003	-58.00	0.0013	2.4
schubfach:go-vs-rs	api	canonicalize	mixed-prod	schubfach-rs*	2.00×	[1.978, 2.023]	0.0002	0.0003	67.48	0.0013	2.4
bd:go-vs-rs	api	canonicalize	mixed-prod	json-canon-rs*	1.30×	[1.286, 1.309]	0.0002	0.0003	25.00	0.0011	1.0
go:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach*	1.41×	[1.402, 1.419]	0.0002	0.0003	-43.96	0.0000	0.8
rs:schubfach-vs-bd	api	canonicalize	nested-mixed	schubfach-rs*	2.19×	[2.114, 2.249]	0.0002	0.0003	-23.61	0.0000	5.6
schubfach:go-vs-rs	api	canonicalize	nested-mixed	schubfach-rs*	2.13×	[2.067, 2.179]	0.0002	0.0003	34.23	0.0000	5.6
bd:go-vs-rs	api	canonicalize	nested-mixed	json-canon-rs*	1.37×	[1.346, 1.390]	0.0002	0.0003	18.98	0.0001	3.2
go:schubfach-vs-bd	api	canonicalize	number-heavy	schubfach*	1.54×	[1.525, 1.553]	0.0002	0.0003	-44.59	0.0002	1.5
rs:schubfach-vs-bd	api	canonicalize	number-heavy	schubfach-rs*	9.08×	[7.899, 9.885]	0.0006	0.0008	-59.55	0.0003	22.7
schubfach:go-vs-rs	api	canonicalize	number-heavy	schubfach-rs*	8.70×	[7.510, 9.476]	0.0002	0.0003	51.93	0.0003	22.7
bd:go-vs-rs	api	canonicalize	number-heavy	json-canon-rs*	1.48×	[1.467, 1.485]	0.0002	0.0003	49.70	0.0001	0.9
go:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach*	1.47×	[1.466, 1.483]	0.0002	0.0003	-66.14	0.0001	1.0
rs:schubfach-vs-bd	api	canonicalize	numeric-boundary	schubfach-rs*	11.50×	[11.355, 11.754]	0.0002	0.0003	-175.11	0.0000	3.3
schubfach:go-vs-rs	api	canonicalize	numeric-boundary	schubfach-rs*	12.94×	[12.769, 13.211]	0.0002	0.0003	152.45	0.0000	3.3
bd:go-vs-rs	api	canonicalize	numeric-boundary	json-canon-rs*	1.66×	[1.649, 1.665]	0.0002	0.0003	97.30	0.0001	0.9
go:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	json-canon*	1.01×	[1.009, 1.019]	0.0004	0.0005	2.23	0.0000	0.9
rs:schubfach-vs-bd	api	canonicalize	rfc-key-sorting	schubfach-rs	1.02×	[1.000, 1.054]	0.1366	0.1569	-0.70	0.0000	3.5
schubfach:go-vs-rs	api	canonicalize	rfc-key-sorting	schubfach-rs*	2.29×	[2.253, 2.329]	0.0002	0.0003	59.46	0.0000	3.5
bd:go-vs-rs	api	canonicalize	rfc-key-sorting	json-canon-rs*	2.21×	[2.148, 2.248]	0.0002	0.0003	44.67	0.0000	4.4
go:schubfach-vs-bd	api	canonicalize	small	schubfach*	1.35×	[1.338, 1.351]	0.0002	0.0003	-49.47	0.0000	0.7

Continued on next page

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
rs:schubfach-vs-bd	api	canonicalize	small	schubfach-rs*	1.92×	[1.870, 1.962]	0.0002	0.0003	-19.05	0.0000	2.8
schubfach:go-vs-rs	api	canonicalize	small	schubfach-rs*	1.89×	[1.871, 1.923]	0.0002	0.0003	50.86	0.0000	2.8
bd:go-vs-rs	api	canonicalize	small	json-canon-rs*	1.33×	[1.303, 1.356]	0.0002	0.0003	13.57	0.0000	4.2
go:schubfach-vs-bd	api	canonicalize	surrogate-pair	json-canon*	1.01×	[1.008, 1.018]	0.0004	0.0005	2.23	0.0000	0.4
rs:schubfach-vs-bd	api	canonicalize	surrogate-pair	json-canon-rs	1.02×	[0.983, 1.061]	0.4681	0.5062	0.35	0.0000	3.0
schubfach:go-vs-rs	api	canonicalize	surrogate-pair	schubfach-rs*	2.01×	[1.924, 2.070]	0.0002	0.0003	23.39	0.0000	7.4
bd:go-vs-rs	api	canonicalize	surrogate-pair	json-canon-rs*	2.02×	[1.980, 2.038]	0.0002	0.0003	58.11	0.0000	3.0
go:schubfach-vs-bd	api	canonicalize	unicode	json-canon	1.00×	[0.998, 1.007]	0.2070	0.2335	0.59	0.0000	0.5
rs:schubfach-vs-bd	api	canonicalize	unicode	schubfach-rs*	1.05×	[1.027, 1.065]	0.0006	0.0008	-2.02	0.0000	1.7
schubfach:go-vs-rs	api	canonicalize	unicode	schubfach-rs*	1.04×	[1.032, 1.050]	0.0002	0.0003	3.94	0.0000	1.7
bd:go-vs-rs	api	canonicalize	unicode	json-canon	1.01×	[0.991, 1.024]	0.4717	0.5069	-0.34	0.0000	0.5
go:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach*	1.72×	[1.709, 1.726]	0.0002	0.0003	-75.40	0.0000	0.5
rs:schubfach-vs-bd	api	canonicalize	verify-whitespace	schubfach-rs*	3.33×	[3.224, 3.430]	0.0002	0.0003	-30.09	0.0000	5.5
schubfach:go-vs-rs	api	canonicalize	verify-whitespace	schubfach-rs*	2.24×	[2.174, 2.291]	0.0002	0.0003	39.12	0.0000	5.5
bd:go-vs-rs	api	canonicalize	verify-whitespace	json-canon-rs*	1.15×	[1.130, 1.173]	0.0002	0.0003	6.97	0.0000	3.8
go:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach*	1.61×	[1.603, 1.618]	0.0002	0.0003	-81.32	0.0108	0.5
rs:schubfach-vs-bd	api	verify	canonical/array-2048	schubfach-rs*	2.93×	[2.910, 2.945]	0.0002	0.0003	-232.73	0.0131	1.2
schubfach:go-vs-rs	api	verify	canonical/array-2048	schubfach-rs*	1.82×	[1.810, 1.833]	0.0002	0.0003	93.40	0.0131	1.2
bd:go-vs-rs	api	verify	canonical/array-2048	json-canon-rs	1.00×	[0.998, 1.006]	0.4755	0.5092	0.34	0.0092	0.3
go:schubfach-vs-bd	api	verify	canonical/array-256	schubfach*	1.52×	[1.512, 1.529]	0.0002	0.0003	-62.21	0.0020	0.8
rs:schubfach-vs-bd	api	verify	canonical/array-256	schubfach-rs*	2.59×	[2.576, 2.607]	0.0002	0.0003	-171.39	0.0016	1.2
schubfach:go-vs-rs	api	verify	canonical/array-256	schubfach-rs*	1.86×	[1.852, 1.877]	0.0002	0.0003	81.30	0.0016	1.2
bd:go-vs-rs	api	verify	canonical/array-256	json-canon-rs*	1.09×	[1.089, 1.098]	0.0002	0.0003	16.47	0.0016	0.5
go:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach*	1.51×	[1.507, 1.522]	0.0002	0.0003	-68.70	0.0000	0.7
rs:schubfach-vs-bd	api	verify	canonical/canonical-minimal	schubfach-rs*	1.40×	[1.328, 1.668]	0.0002	0.0003	-3.56	0.0001	19.9
schubfach:go-vs-rs	api	verify	canonical/canonical-minimal	schubfach	1.06×	[0.850, 1.121]	0.3171	0.3503	-0.52	0.0000	0.7
bd:go-vs-rs	api	verify	canonical/canonical-minimal	json-canon-rs*	1.01×	[1.005, 1.029]	0.0204	0.0251	1.08	0.0000	2.3
go:schubfach-vs-bd	api	verify	canonical/control-escapes	json-canon*	1.05×	[1.040, 1.053]	0.0002	0.0003	6.24	0.0000	0.5
rs:schubfach-vs-bd	api	verify	canonical/control-escapes	json-canon-rs*	1.82×	[1.527, 1.941]	0.0002	0.0003	3.85	0.0000	3.1
schubfach:go-vs-rs	api	verify	canonical/control-escapes	schubfach-rs*	1.17×	[1.101, 1.399]	0.0002	0.0003	1.45	0.0002	20.7
bd:go-vs-rs	api	verify	canonical/control-escapes	json-canon-rs*	2.04×	[2.008, 2.070]	0.0002	0.0003	56.78	0.0000	3.1
go:schubfach-vs-bd	api	verify	canonical/deep-64	schubfach	1.01×	[1.001, 1.010]	0.0420	0.0508	-1.02	0.0001	0.6
rs:schubfach-vs-bd	api	verify	canonical/deep-64	json-canon-rs*	1.19×	[1.133, 1.272]	0.0004	0.0005	2.30	0.0001	1.1
schubfach:go-vs-rs	api	verify	canonical/deep-64	schubfach-rs*	1.26×	[1.181, 1.323]	0.0004	0.0005	3.83	0.0013	12.2
bd:go-vs-rs	api	verify	canonical/deep-64	json-canon-rs*	1.51×	[1.504, 1.522]	0.0002	0.0003	58.85	0.0001	1.1
go:schubfach-vs-bd	api	verify	canonical/deep	json-canon	1.00×	[0.996, 1.007]	0.7143	0.7304	0.17	0.0003	0.9
rs:schubfach-vs-bd	api	verify	canonical/deep	json-canon-rs*	1.02×	[0.996, 1.032]	0.0298	0.0363	1.01	0.0002	0.7
schubfach:go-vs-rs	api	verify	canonical/deep	schubfach-rs*	1.24×	[1.223, 1.267]	0.0002	0.0003	11.79	0.0008	3.4
bd:go-vs-rs	api	verify	canonical/deep	json-canon-rs*	1.26×	[1.255, 1.268]	0.0002	0.0003	35.05	0.0002	0.7
go:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach*	1.64×	[1.631, 1.644]	0.0002	0.0003	-100.61	0.0000	0.6
rs:schubfach-vs-bd	api	verify	canonical/escaped-key-order	schubfach-rs*	1.62×	[1.521, 1.956]	0.0002	0.0003	-4.99	0.0001	21.3
schubfach:go-vs-rs	api	verify	canonical/escaped-key-order	schubfach-rs*	1.13×	[1.066, 1.372]	0.0002	0.0003	1.11	0.0001	21.3
bd:go-vs-rs	api	verify	canonical/escaped-key-order	json-canon-rs*	1.15×	[1.128, 1.165]	0.0002	0.0003	7.60	0.0000	3.4
go:schubfach-vs-bd	api	verify	canonical/large	schubfach*	1.75×	[1.740, 1.757]	0.0002	0.0003	-75.44	0.0496	0.5
rs:schubfach-vs-bd	api	verify	canonical/large	schubfach-rs*	3.98×	[3.882, 4.042]	0.0002	0.0003	-88.16	0.1912	4.1
schubfach:go-vs-rs	api	verify	canonical/large	schubfach-rs*	1.94×	[1.893, 1.972]	0.0002	0.0003	39.78	0.1912	4.1
bd:go-vs-rs	api	verify	canonical/large	json-canon*	1.17×	[1.162, 1.179]	0.0002	0.0003	-18.90	0.1529	1.0
go:schubfach-vs-bd	api	verify	canonical/long-string	json-canon	1.00×	[0.985, 1.015]	0.9752	0.9783	0.01	0.0017	2.7
rs:schubfach-vs-bd	api	verify	canonical/long-string	json-canon-rs*	1.08×	[1.047, 1.096]	0.0002	0.0003	3.21	0.0003	2.0
schubfach:go-vs-rs	api	verify	canonical/long-string	schubfach-rs*	4.09×	[4.036, 4.223]	0.0004	0.0005	70.03	0.0006	3.7
bd:go-vs-rs	api	verify	canonical/long-string	json-canon-rs*	4.42×	[4.351, 4.490]	0.0002	0.0003	49.83	0.0003	2.0
go:schubfach-vs-bd	api	verify	canonical/medium	schubfach*	1.70×	[1.688, 1.704]	0.0002	0.0003	-99.20	0.0020	0.7
rs:schubfach-vs-bd	api	verify	canonical/medium	schubfach-rs*	3.34×	[3.160, 3.457]	0.0002	0.0003	-42.89	0.0144	9.4
schubfach:go-vs-rs	api	verify	canonical/medium	schubfach-rs*	1.78×	[1.670, 1.839]	0.0002	0.0003	14.51	0.0144	9.4
bd:go-vs-rs	api	verify	canonical/medium	json-canon*	1.11×	[1.103, 1.113]	0.0002	0.0003	-20.03	0.0027	0.6
go:schubfach-vs-bd	api	verify	canonical/mixed-prod	schubfach*	1.33×	[1.317, 1.333]	0.0002	0.0003	-41.20	0.0012	1.2
rs:schubfach-vs-bd	api	verify	canonical/mixed-prod	schubfach-rs*	1.94×	[1.779, 2.014]	0.0002	0.0003	-14.36	0.0065	11.5
schubfach:go-vs-rs	api	verify	canonical/mixed-prod	schubfach-rs*	1.90×	[1.732, 1.964]	0.0002	0.0003	13.53	0.0065	11.5
bd:go-vs-rs	api	verify	canonical/mixed-prod	json-canon-rs*	1.29×	[1.285, 1.298]	0.0002	0.0003	44.84	0.0009	0.9
go:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach*	1.41×	[1.402, 1.415]	0.0002	0.0003	-52.88	0.0000	0.4
rs:schubfach-vs-bd	api	verify	canonical/nested-mixed	schubfach-rs*	2.23×	[2.157, 2.291]	0.0002	0.0003	-22.93	0.0000	5.6
schubfach:go-vs-rs	api	verify	canonical/nested-mixed	schubfach-rs*	2.20×	[2.122, 2.242]	0.0002	0.0003	37.29	0.0000	5.6
bd:go-vs-rs	api	verify	canonical/nested-mixed	json-canon-rs*	1.39×	[1.358, 1.405]	0.0002	0.0003	18.57	0.0001	3.4

Continued on next page

Pair	Track	Mode	Workload	Winner	Speedup	CI_{95}	p	p_{adj}	d	Noise (ms)	MDE (%)
go:schubfach-vs-bd	api	verify	canonical/number-heavy	schubfach*	1.54×	[1.531, 1.553]	0.0002	0.0003	-51.34	0.0001	1.1
rs:schubfach-vs-bd	api	verify	canonical/number-heavy	schubfach-rs*	10.22×	[9.954, 10.753]	0.0002	0.0003	-25.03	0.0000	4.1
schubfach:go-vs-rs	api	verify	canonical/number-heavy	schubfach-rs*	9.80×	[9.587, 9.976]	0.0002	0.0003	139.82	0.0000	4.1
bd:go-vs-rs	api	verify	canonical/number-heavy	json-canon-rs*	1.48×	[1.392, 1.509]	0.0002	0.0003	13.06	0.0007	6.4
go:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach*	1.49×	[1.476, 1.502]	0.0002	0.0003	-34.75	0.0001	1.1
rs:schubfach-vs-bd	api	verify	canonical/numeric-boundary	schubfach-rs*	11.62×	[11.449, 12.116]	0.0004	0.0005	-191.07	0.0000	4.8
schubfach:go-vs-rs	api	verify	canonical/numeric-boundary	schubfach-rs*	13.33×	[13.117, 13.823]	0.0002	0.0003	142.40	0.0000	4.8
bd:go-vs-rs	api	verify	canonical/numeric-boundary	json-canon-rs*	1.71×	[1.693, 1.720]	0.0002	0.0003	46.91	0.0001	0.7
go:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	json-canon	1.00×	[0.998, 1.008]	0.3087	0.3422	0.45	0.0000	0.9
rs:schubfach-vs-bd	api	verify	canonical/rfc-key-sorting	json-canon-rs	1.02×	[0.988, 1.056]	0.2955	0.3299	0.48	0.0001	5.1
schubfach:go-vs-rs	api	verify	canonical/rfc-key-sorting	schubfach-rs*	2.09×	[2.028, 2.136]	0.0002	0.0003	35.84	0.0001	5.1
bd:go-vs-rs	api	verify	canonical/rfc-key-sorting	json-canon-rs*	2.13×	[2.073, 2.174]	0.0002	0.0003	36.97	0.0001	5.1
go:schubfach-vs-bd	api	verify	canonical/small	schubfach*	1.36×	[1.350, 1.367]	0.0002	0.0003	-43.17	0.0000	1.0
rs:schubfach-vs-bd	api	verify	canonical/small	schubfach-rs*	1.90×	[1.788, 1.978]	0.0002	0.0003	-12.64	0.0001	9.2
schubfach:go-vs-rs	api	verify	canonical/small	schubfach-rs*	1.80×	[1.687, 1.855]	0.0002	0.0003	15.05	0.0001	9.2
bd:go-vs-rs	api	verify	canonical/small	json-canon-rs*	1.28×	[1.257, 1.313]	0.0002	0.0003	10.77	0.0000	4.6
go:schubfach-vs-bd	api	verify	canonical/surrogate-pair	json-canon	1.00×	[0.998, 1.006]	0.3699	0.4058	0.42	0.0000	0.7
rs:schubfach-vs-bd	api	verify	canonical/surrogate-pair	json-canon-rs	1.01×	[0.987, 1.048]	0.5279	0.5578	0.31	0.0000	2.5
schubfach:go-vs-rs	api	verify	canonical/surrogate-pair	schubfach-rs*	2.07×	[2.000, 2.115]	0.0002	0.0003	32.89	0.0000	5.7
bd:go-vs-rs	api	verify	canonical/surrogate-pair	json-canon-rs*	2.09×	[2.063, 2.114]	0.0002	0.0003	66.74	0.0000	2.5
go:schubfach-vs-bd	api	verify	canonical/unicode	schubfach	1.00×	[1.000, 1.008]	0.0910	0.1076	-0.80	0.0000	0.6
rs:schubfach-vs-bd	api	verify	canonical/unicode	schubfach-rs	1.02×	[0.923, 1.068]	0.6383	0.6612	-0.28	0.0002	13.1
schubfach:go-vs-rs	api	verify	canonical/unicode	schubfach	1.00×	[0.957, 1.107]	0.9948	0.9948	-0.01	0.0000	0.6
bd:go-vs-rs	api	verify	canonical/unicode	json-canon*	1.02×	[1.012, 1.024]	0.0002	0.0003	-2.36	0.0000	0.6
go:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach*	1.71×	[1.703, 1.714]	0.0004	0.0005	-132.00	0.0000	0.4
rs:schubfach-vs-bd	api	verify	canonical/verify-whitespace	schubfach-rs*	3.41×	[3.201, 3.527]	0.0002	0.0003	-30.70	0.0000	9.2
schubfach:go-vs-rs	api	verify	canonical/verify-whitespace	schubfach-rs*	2.33×	[2.173, 2.398]	0.0002	0.0003	25.30	0.0000	9.2
bd:go-vs-rs	api	verify	canonical/verify-whitespace	json-canon-rs*	1.16×	[1.152, 1.187]	0.0002	0.0003	9.38	0.0000	3.1
go:schubfach-vs-bd	api	verify	noncanonical/control-escapes	json-canon*	1.04×	[1.030, 1.041]	0.0004	0.0005	5.38	0.0000	0.9
go:schubfach-vs-bd	api	verify	noncanonical/escaped-key-order	schubfach*	1.61×	[1.605, 1.622]	0.0002	0.0003	-66.53	0.0000	0.7
go:schubfach-vs-bd	api	verify	noncanonical/large	schubfach*	1.73×	[1.720, 1.739]	0.0002	0.0003	-76.99	0.0777	0.8
go:schubfach-vs-bd	api	verify	noncanonical/medium	schubfach*	1.71×	[1.700, 1.719]	0.0002	0.0003	-73.52	0.0021	0.8
go:schubfach-vs-bd	api	verify	noncanonical/mixed-prod	schubfach*	1.32×	[1.309, 1.331]	0.0002	0.0003	-25.37	0.0007	0.7
go:schubfach-vs-bd	api	verify	noncanonical/nested-mixed	schubfach*	1.42×	[1.413, 1.426]	0.0002	0.0003	-62.44	0.0000	0.6
go:schubfach-vs-bd	api	verify	noncanonical/number-heavy	schubfach*	1.55×	[1.542, 1.563]	0.0002	0.0003	-48.98	0.0001	1.1
go:schubfach-vs-bd	api	verify	noncanonical/numeric-boundary	schubfach*	1.49×	[1.470, 1.510]	0.0002	0.0003	-22.10	0.0001	1.4
go:schubfach-vs-bd	api	verify	noncanonical/rfc-key-sorting	json-canon	1.00×	[0.995, 1.009]	0.6203	0.6447	0.22	0.0000	0.9
go:schubfach-vs-bd	api	verify	noncanonical/small	schubfach*	1.35×	[1.347, 1.361]	0.0002	0.0003	-45.94	0.0000	0.7
go:schubfach-vs-bd	api	verify	noncanonical/surrogate-pair	json-canon	1.00×	[0.998, 1.008]	0.4185	0.4575	0.41	0.0000	0.7
go:schubfach-vs-bd	api	verify	noncanonical/unicode	schubfach	1.01×	[0.999, 1.010]	0.1038	0.1218	-0.76	0.0000	0.5
go:schubfach-vs-bd	api	verify	noncanonical/verify-whitespace	schubfach*	1.73×	[1.725, 1.743]	0.0002	0.0003	-70.65	0.0000	0.5

*Significant after Benjamini-Hochberg FDR correction at $q = 0.05$.
d = Cohen's d: $(\bar{x}_A - \bar{x}_B)/s_p$; negative = impl A faster. Track: e2e = end-to-end CLI.
Noise = CV × mean of faster impl. MDE = minimum detectable effect at 80% power (z-approx).
Rust allocs/op and B/op measured via counting global allocator (§8).

B Reproducibility Checklist

This appendix documents the information required to reproduce the benchmark results reported in this paper.

Software Versions

- Go 1.25.6 (`go version go1.25.6 linux/amd64`)
- Rust 1.94.0 (`rustc 1.94.0`)
- Node.js v23.3.0 (V8 12.9.202.28-node.11) — oracle generation
- Linux 6.8.0-101-generic (Ubuntu)

Hardware Specification

- CPU: Intel Core i7-12700K (8 P-cores at 3.6–5.0 GHz, 4 E-cores at 2.7–3.8 GHz)
- RAM: 64 GB DDR4
- Architecture: x86-64
- CPU governor: `performance`

Oracle Vector Checksums (SHA-256)

- `golden_vectors.csv` (54,445 vectors):
  `593bdecbe0dccbc182bc3baf570b716887db25739fc61b7808764ecb966d5636`
- `golden_stress_vectors.csv` (231,917 vectors):
  `287d21ac87e5665550f1baf86038302a0afc67a74a020dffb872f1a93b26d410`

Reproduction Commands

```
git clone <repository-url> && cd jcs-bench-lab
go run ./cmd/lab setup
go run ./cmd/lab conformance
go run ./cmd/lab bench-api -count 10
go run ./cmd/lab bench-cli -repeats 15
go run ./cmd/lab fuzz -cases 2000
go run ./cmd/lab stats -resamples 5000
go run ./cmd/lab gate
go run scripts/generate-paper-tables.go
```

Build Configuration

- Go implementations: default `go build` flags
- Rust implementations: `opt-level=3`, `lto=true`, `codegen-units=1`, `strip=true`
- Rust API benchmarks use a counting global allocator (`std::alloc::GlobalAlloc`) to measure per-operation allocation bytes and counts

Data Availability

All raw benchmark data, statistical reports, oracle vectors, and the benchmark infrastructure are archived at Zenodo.[3]

[3]Zenodo DOI to be inserted for camera-ready version.